Universitat
de les Illes Balears

TREBALL DE FI DE GRAU

WAVE LAUNCHER: A LOCAL MINECRAFT SERVER MANAGER

Nahuel Vazquez

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Academic year 2025-26

WAVE LAUNCHER: A LOCAL MINECRAFT SERVER MANAGER

Nahuel Vazquez

Treball de Fi de Grau

Escola Politècnica Superior

Universitat de les Illes Balears

Academic year 2025-26

Paraules clau del treball: TFG, memòria, LaTeX, server, minecraft, mods, local

Tutor: Maria Francesca Roig Maimó

Gràcies a tots els professors que m'han ajudat a arribar a aquesta fita dels meus estudis.

SUMARI

Sumari iii

Acronyms vii

Summary ix

1 Introduction 1
- 1.1 Contextualization . . . 1
- 1.2 Motivation . . . 1
- 1.3 Objective . . . 2
- 1.4 Scope . . . 2
  - 1.4.1 Server instance management . . . 3
  - 1.4.2 Mod-loader and mod management . . . 3
  - 1.4.3 Runtime and network management . . . 4

2 Alternatives 5
- 2.1 Free server managers . . . 5
- 2.2 AMP . . . 6
- 2.3 Comparative analysis . . . 6

3 Project management 9
- 3.1 Origin of the project . . . 9
- 3.2 Development method . . . 10
- 3.3 Phases and evolution . . . 10
  - 3.3.1 Project and architecture setup . . . 10
  - 3.3.2 Services and external integrations . . . 11
  - 3.3.3 User-interface implementation . . . 11
- 3.4 Risk management . . . 12
- 3.5 Assessment of the process . . . 12

4 System analysis and design 13
- 4.1 Users and operating context . . . 13
- 4.2 Quality requirements . . . 13
- 4.3 Architectural design . . . 15
  - 4.3.1 Choice of architectural style . . . 15
  - 4.3.2 Solution projects . . . 15
- 4.4 Domain model . . . 16

4.4.1 Server aggregate . . . 16
4.4.2 Version and installation types . . . 17
4.4.3 Mod representation . . . 17
4.5 Ports and adapters . . . 17
4.5.1 Input ports . . . 17
4.5.2 Intermediate services . . . 17
4.5.3 Output ports . . . 18
4.6 Persistence and directory design . . . 18
4.7 Design of the principal workflows . . . 19
4.7.1 Server creation and editing . . . 19
4.7.2 Java management . . . 21
4.7.3 Mod-loader management . . . 21
4.7.4 Mod discovery and dependency resolution . . . 21
4.7.5 Execution and port mapping . . . 22
4.8 User-interface design . . . 23
4.9 Technology selection . . . 24
4.9.1 C#, .NET and MAUI . . . 24
4.9.2 Libraries . . . 25
4.9.3 Development environment . . . 25
4.10 Design limitations . . . 25

5 Implementation **27**
5.1 Application composition . . . 27
5.2 Server persistence and file management . . . 28
5.2.1 JSON repositories . . . 28
5.2.2 Paths and cleanup . . . 28
5.3 Minecraft catalogue and installation . . . 28
5.4 Java runtime management . . . 29
5.4.1 Supplier adapters . . . 29
5.4.2 Selection and removal . . . 29
5.5 Forge and Fabric integration . . . 29
5.6 Mod provider integration . . . 30
5.6.1 Common supplier contract . . . 30
5.6.2 Modrinth . . . 31
5.6.3 CurseForge . . . 31
5.7 Dependency installation and migration . . . 31
5.8 Server creation and update orchestration . . . 32
5.9 Process execution and console . . . 34
5.10 Automatic port mapping . . . 35
5.11 MAUI user interface . . . 36
5.11.1 Shell and pages . . . 36
5.11.2 Reusable forms . . . 37
5.11.3 Feedback and pagination . . . 39
5.12 Error handling . . . 39

6 Validation **41**
6.1 Environment and compatibility matrix . . . 41

6.2 Functional validation 42
6.2.1 Testing method 42
6.2.2 End-to-end procedure and results 42
6.2.3 Requirement coverage 43
6.3 Comparison with manual server setup 44
6.3.1 Test procedure 44
6.3.2 Manual creation across participant backgrounds 45
6.3.3 Wave Launcher use across participant backgrounds 47
6.3.4 Manual setup compared with Wave Launcher 48
6.4 Known defects and limitations observed during testing 49

7 Conclusions and future work **51**
7.1 Conclusions 51
7.2 Future work 52
7.2.1 Extend the empirical evaluation 52
7.2.2 Additional platforms 52
7.2.3 Modpack support 52
7.2.4 World migration with MCA Selector 53
7.2.5 Whitelist management 53
7.2.6 Network diagnostics 53
7.2.7 Distribution and community development 53

A User guide **55**
A.1 Initial configuration 55
A.2 Creating a server 55
A.3 Editing and migrating 55
A.4 Starting and stopping 59
A.5 Removing content 59

B Repository structure **61**

Bibliografia **63**

Acronyms

API	Application Programming Interface
DTO	Data Transfer Object
EULA	End User Licence Agreement
HTTP	Hypertext Transfer Protocol
JAR	Java Archive
MOTD	Message of the Day
NAT	Network Address Translation
NAT-PMP	NAT Port Mapping Protocol
REST	Representational State Transfer
TCP	Transmission Control Protocol
UI	User Interface
UPnP	Universal Plug and Play
XML	Extensible Markup Language

SUMMARY

This report describes the design and implementation of Wave Launcher, a local application for creating and managing self-hosted Minecraft: Java Edition servers. Existing management tools simplify some parts of server configuration and execution. However, they often leave the user to select a Java runtime, organize mod files, solve compatibility problems and configure external network access. For casual players who want a private server without paying for commercial hosting, these tasks can be a considerable obstacle.

Wave Launcher addresses this problem through a graphical interface that coordinates the server software, Java runtime, mod loader and installed mods. It obtains these components through the APIs of their respective distributors and keeps track of the compatibility relationships between their versions. The application supports Java runtimes from Mojang and Eclipse Adoptium, the Forge and Fabric mod loaders, and mods from Modrinth and CurseForge. It also assists with network configuration and server updates, reducing the need to manipulate the underlying files directly.

The application is implemented in C# using .NET MAUI 10, which provides a cross-platform framework while using native controls on each supported platform. Its internal organization is based on hexagonal architecture, with the user interface, application logic and external integrations separated into distinct responsibilities. Because all of these components execute locally within the same application, the design does not introduce a separate web API between the interface and the application services. The current implementation targets Windows, while the selected framework and architecture leave open the possibility of supporting additional platforms in future work.

CHAPTER 1

INTRODUCTION

1.1 Contextualization

Minecraft supports both single-player and multiplayer sessions. In multiplayer, the game running on each player's computer acts as a client and connects to a server that maintains the shared world and coordinates player activity. Public servers are suitable for players seeking established communities or minigames, but they do not provide an environment controlled exclusively by a particular group of friends. Players seeking a private environment must instead obtain a server from a hosting provider or run one on their own hardware.

The choice is therefore a trade-off between convenience, cost and control. A hosting provider takes care of the underlying infrastructure and usually includes a management interface, but it requires a recurring payment and stores the server files on a third-party system. Self-hosting avoids this cost and gives the player direct control over the server and its data. In return, the player becomes responsible for installation, configuration, networking and maintenance.

Minecraft client launchers provide interfaces for a range of users and technical requirements. Equivalent tools for self-hosted servers exist, but their primary function is generally to administer and execute server software that has already been selected and configured. As examined in Chapter 2, casual players have fewer options that also manage the dependencies and network configuration required throughout the complete server lifecycle.

1.2 Motivation

In practice, creating a Minecraft: Java Edition server manually is not a single installation task. Several components have to be obtained and configured in the right order. The user needs the appropriate server software and a compatible Java runtime, and must also choose startup parameters and edit text-based configuration files. Since every com-

ponent has its own version requirements, a configuration that works for one Minecraft release may fail with another.

Modded servers extend this dependency chain. In addition to the base server and Java runtime, the user must select a compatible mod loader and obtain suitable versions of every mod and dependency. The ability to download or execute these files does not by itself resolve whether they support the selected Minecraft and loader versions. Compatibility failures may therefore become visible only when the server is started, leaving the user to identify and replace the conflicting files manually.

Local execution also does not make the server reachable from outside the user's network. The player must normally configure the household router so that incoming connections are directed to the server. This requires familiarity with ports, local addresses and router administration, and may be an entry barrier for casual players.

The same coordination is required during maintenance. Updating Minecraft may require a different Java runtime, a new mod-loader release and compatible updates for the installed mods. Performing these changes directly on the server files is repetitive and creates opportunities for incompatible or obsolete components to remain in the installation.

1.3 Objective

The objective of Wave Launcher is to enable casual players to create and manage private, self-hosted Minecraft: Java Edition servers on Windows without paying recurring hosting fees or requiring server-administration knowledge. In this context, a casual player is not expected to install Java runtimes, manipulate server files, resolve version dependencies or configure network ports manually.

To meet this objective, the application must provide a graphical workflow for creating, configuring, starting, stopping, updating and removing server instances. It must coordinate the server software, Java runtime, mod loader and installed mods, selecting mutually compatible versions where the available provider metadata permits this. Mod discovery, installation, removal and updating must be exposed as application operations rather than as direct file management tasks.

The application must also assist with making a server reachable outside the local network and communicate any conditions that cannot be configured automatically. These capabilities are intended to reduce the user's role to selecting the desired server and gameplay options while the application manages their technical consequences. Wave Launcher is not intended to replace general-purpose administration panels or commercial hosting; it addresses the narrower use case of a casual player operating a private Minecraft server from their own computer.

1.4 Scope

The scope of Wave Launcher is defined by the following functional requirements. All requirements are mandatory for the application to satisfy its objective. The identifiers assigned to them provide a stable reference for the design, implementation and validation presented in subsequent chapters.

1.4.1 Server instance management

FR-01: View server instances. The user must be able to view all created server instances. Each item must display enough information to distinguish it from the others, including the server name, icon, execution status and message of the day (MOTD).

FR-02: Create server instances. The user must be able to create a server instance through a graphical interface. The interface must allow the user to specify the server name, icon and Minecraft version, including snapshot and pre-release versions; configure the server properties, such as game mode, difficulty and view distance; define launch flags; and approve the Minecraft End User Licence Agreement (EULA). After submission, the application must create the server directory hierarchy and automatically download the files required for execution.

FR-03: Delete server instances. The user must be able to delete an existing server instance through the graphical interface. The application must remove all files belonging to that instance.

FR-04: Execute server instances. The user must be able to start a server instance through the graphical interface. The application must redirect the server output to the interface so that its logs can be inspected while it is running.

FR-05: Send server commands. The user must be able to submit commands to a running server through the graphical interface.

FR-06: Stop server instances. The user must be able to stop a running server instance through the graphical interface.

1.4.2 Mod-loader and mod management

FR-07: Add a mod loader. The user must be able to add a mod loader to an existing server instance through the graphical interface. The application must support at least Forge and Fabric and must allow the loader and its specific version to be selected. A server instance must not have more than one mod loader installed at a time.

FR-08: Change a mod-loader version. The user must be able to upgrade or downgrade the mod loader installed in a server instance through the graphical interface.

FR-09: Remove a mod loader. The user must be able to remove the mod loader installed in a server instance through the graphical interface. The application must remove the loader files and all mods installed for that loader.

FR-10: Add mods. The user must be able to browse and add mods to an existing server instance through the graphical interface. The application must support Modrinth and, subject to obtaining the required API credentials, CurseForge. Search results must be restricted to releases compatible with the mod loader installed in the selected server instance.

FR-11: Remove mods. The user must be able to remove installed mods from a server instance through the graphical interface.

FR-12: Change the Minecraft version. The user must be able to upgrade or downgrade the Minecraft version of a server instance through the graphical interface. The application must obtain the appropriate server files and determine the corresponding changes required for the installed mod loader and mods. If no compatible version of a required component is available, the application must notify the user and abort the migration without leaving the instance partially updated.

1.4.3 Runtime and network management

FR-13: Manage Java runtimes. The user must be able to browse the available Java runtime versions and install or uninstall them through the graphical interface. Server instances must be configured to use an installed runtime compatible with their Minecraft version.

FR-14: Configure network ports. When a server is started, the application must open or forward the network ports required to make it accessible. Before doing so, it must check whether the selected port is already in use by another instance. If a conflict exists, the application must abort startup and notify the user.

CHAPTER 2

ALTERNATIVES

I assessed the alternatives from the perspective of a casual player who wants to host a private Minecraft: Java Edition server on a Windows computer. In this comparison, that player is not expected to know how to select Java runtimes, manipulate server files, resolve compatibility among Minecraft versions, mod loaders and mods, or configure port forwarding. The comparison uses the functionality described in each application's official documentation, consulted on 9 August 2026. It focuses on documented capabilities and does not attempt to rank subjective qualities such as visual design.

The selected alternatives are Crafty Controller, AMP, MC Server Soft, Fork and MCSManager. All five provide a graphical interface from which servers can be created or imported, configured and executed. They also support multiple server instances and can run modded server software. However, the ability to execute a server that already contains mods must be distinguished from managing the mod lifecycle: discovering mods, selecting compatible releases, installing their dependencies and keeping them updated.

2.1 Free server managers

Crafty Controller is a web-based administration interface with server creation, file management, backups, metrics, scheduled tasks and multi-user roles [1]. It supports Windows and is distributed under the GNU GPLv3 [2]. Its installation documentation nevertheless states that Java must be installed before a Minecraft server can be started [3]. Although Crafty can install and execute modded server types, its documented management facilities are centred on server files rather than on browsing individual mods and resolving their dependencies. Crafty's own website states that getting started requires "a bit of technical knowledge and a desire to learn" [4]. This is a reasonable requirement for novice server administrators, but it still represents a potential barrier for the target user defined in this project, who may neither possess that knowledge nor wish to acquire it merely to host a private server.

MC Server Soft is a free Windows server wrapper that provides server importing, Forge and Fabric installers, backups, scheduling, configuration editors and management of multiple servers [5]. Its stated prerequisites include an installed Java release, a router capable of port forwarding and basic knowledge of Minecraft server administration [6]. Server software must be obtained or selected separately, and its documentation directs the user to the respective external sources [7]. Its plugin manager does not constitute equivalent lifecycle management for Forge or Fabric.

Fork is a free, open-source Windows application explicitly presented as a Minecraft server manager for casual players. It creates and imports servers, edits their configuration and manages multiple instances. It also contains a plugin manager, but does not document an equivalent browser and compatibility resolver for mods. Fork detects Java installations and warns when one is not available; the user must still provide the required runtime. Its own frequently asked questions also instruct users to forward the server port when access from outside the home network is required [8].

MCSManager is an open-source web control panel, licensed under Apache 2.0, that runs on Windows and Linux. It provides multi-user and distributed management of Minecraft, Steam and other game servers [9]. Its manual workflow requires the user to supply a server core and startup command. The documentation also assumes that an appropriate Java runtime has already been installed and provides a version table for users to select it themselves [10, 11]. Thus, its broad deployment and multi-machine facilities are useful to administrators, but they do not remove the mod-compatibility and networking tasks identified in this project's target scenario.

2.2 AMP

AMP differs from the other alternatives because its current Store can browse, install, update, disable and remove Minecraft content. It filters content by game version and mod loader, and supports individual mods and modpacks from Modrinth as well as CurseForge integration [12]. This makes AMP the closest alternative to the mod-management component proposed by Wave Launcher.

AMP is proprietary and has no permanent free edition. A licence is not required to install and inspect the application, but continued use requires a one-time purchase. At the time of the comparison, the Professional licence costs USD 20 and supports 15 application instances, unlimited panel users, multi-system management and webhooks. The Advanced licence costs USD 40 and supports 50 instances, adding analytics and geographic filtering, custom branding, OpenID Connect single sign-on, deployment templates and priority support [13]. AMP supports several games in addition to Minecraft, and its Professional edition is described as being intended for power users managing multiple systems or a larger number of game servers. These capabilities are valuable in larger installations, but add cost and scope that are not required by a casual player seeking to manage one private Minecraft server.

2.3 Comparative analysis

Table 2.1 summarizes the criteria that directly follow from the target user's needs. "Manual" indicates that the application can store or execute the corresponding files but

leaves their acquisition, selection or network configuration to the user. “Integrated” is reserved for functionality that the application performs through its own interface.

Table 2.1: Comparison of Minecraft server management alternatives.

	Crafty	AMP	MCSS	Fork	MCSManager
Windows support	Yes	Yes	Yes	Yes	Yes
Multiple servers	Yes	Yes	Yes	Yes	Yes
No purchase required	Yes	No	Yes	Yes	Yes
Open source	Yes	No	No	Yes	Yes
Modded server execution	Yes	Yes	Yes	Yes	Yes
Integrated mod lifecycle	No	Yes	No	No	No
Java acquisition/selection	Manual	Partial	Manual	Manual	Manual
External network access	Manual	Manual	Manual	Manual	Manual
Minecraft-only scope	Yes	No	Yes	Yes	No

This comparison does not identify one application as the best choice in every situation. AMP is the strongest existing option when integrated mod management, administration of several games and multi-user control justify the licence cost. The free alternatives are suitable when the user is willing to manage Java, mod files, compatibility and network access manually. Wave Launcher occupies the narrower space between these approaches. It is a free, open-source and Minecraft-specific desktop application that manages the required Java runtime, assists with external connectivity, and treats mod discovery, compatibility, installation and updating as central operations. Its intended advantage is not a longer list of administrative features, but a reduction in the manual tasks that commonly prevent casual players from self-hosting.

CHAPTER 3

PROJECT MANAGEMENT

3.1 Origin of the project

The project began with a search for a practical problem that had not been solved satisfactorily for non-technical users. I considered several possible applications based on difficulties I had encountered myself or heard about from other people. The idea for Wave Launcher appeared when acquaintances asked me for help setting up a Minecraft server because they could not complete the process on their own. No single step was especially obscure. The difficulty came from having to combine a server distribution, a suitable Java runtime, configuration files, a mod loader, compatible mods and router settings. This experience gave the project both a concrete target user and a useful objective.

An initial technology study was carried out before a tutor was sought. C# was selected partly because the project offered an opportunity to improve existing knowledge of the language. A multi-platform user interface was also an early objective, which led to the selection of .NET MAUI. The first release was nevertheless scoped to Windows because this is where most casual Minecraft: Java Edition players use the game. It was also the platform available for development and testing. MAUI preserves the possibility of reusing the interface on other platforms, but supporting them was not part of the completed scope because process execution and device-information access would require additional platform-specific implementations.

Before formally committing to the topic, a one-month feasibility period was used to explore the framework, the available provider interfaces and the likely complexity of the server-management operations. At the end of that period, the project was considered feasible and development continued as the final degree project.

This exploratory beginning was useful because several external dependencies were not under the project's control. Mojang, Eclipse Adoptium, Forge, Fabric, Modrinth and CurseForge each publish different data and use different release models. Confirming that enough metadata could be obtained was necessary before a graphical workflow could be promised. The feasibility work also showed that the program could own its

Java installations and server files instead of depending entirely on software configured elsewhere on the computer.

3.2 Development method

Development followed an iterative and incremental process. At the start of each working week, I selected a milestone that seemed achievable in approximately 10 to 20 hours. Each milestone had a concrete result, such as defining a group of ports, integrating an external catalogue, implementing a management operation or completing a user-interface workflow. Small or familiar tasks, particularly integrations similar to one already completed, were grouped together. Less predictable work was given a complete iteration.

The process was deliberately lightweight because the project had one developer. A Scrum process would have introduced ceremonies and roles that could not serve their normal purpose in a one-person project. Instead, the weekly milestone provided a short feedback cycle without pretending that a team process was being followed. The high-level schedule was represented in a Gantt diagram, while daily work was tracked with the Todo Tree extension for Visual Studio Code. Todo Tree collected pending interfaces, services, defects and general improvements directly from annotations in the source tree.

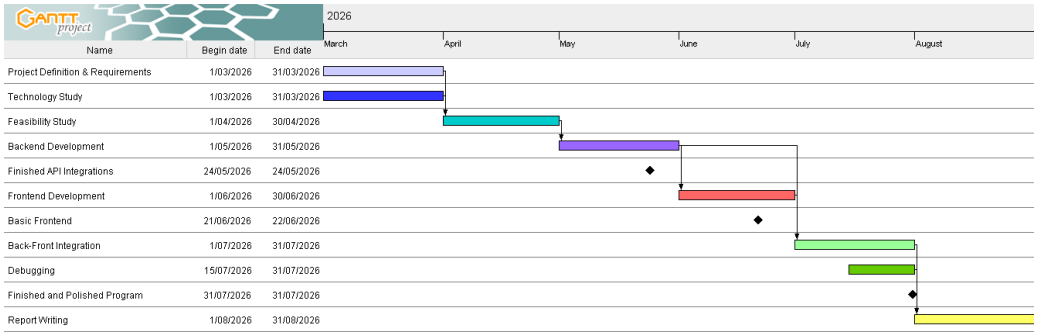

Figure 3.1: High-level Gantt diagram of the project phases.

The Gantt plan separated the work into project definition and requirements, technology study, back-end implementation, user-interface implementation, integration and correction, and report writing. Notes for the report were collected throughout development so that design decisions and implementation difficulties would not have to be reconstructed only at the end. Git and GitHub were used for version control and remote backup. No branching strategy was necessary for a single developer; a commit was normally pushed after a significant milestone had reached a stable state.

3.3 Phases and evolution

3.3.1 Project and architecture setup

The first implementation phase established the solution and its C# projects. The user interface, application contracts, infrastructure implementations and domain model

were separated from the beginning. This setup took approximately two weeks because both .NET MAUI and the architectural style were new in practice. Much of the time was consequently spent learning how MAUI applications are composed, how dependencies cross project boundaries and how asynchronous operations should be exposed to view models.

Starting with boundaries rather than concrete integrations had a direct effect on later work. The interfaces in the application project described the operations needed by the functional requirements, and the infrastructure project could then implement those contracts with file-system, process and HTTP details. It was still possible to revise a contract when the interface revealed a missing use case, but provider-specific data did not become the public vocabulary of the complete application.

3.3.2 Services and external integrations

After the project structure was stable, the input services and output ports were defined. This stage lasted approximately three months and included the study and integration of the external sources. Java distribution required more investigation than initially expected. Microsoft OpenJDK was considered, but it did not provide the catalogue interface needed by the application, so the supported suppliers were limited to Eclipse Adoptium and Mojang. Both major mod platforms, Modrinth and CurseForge, provide APIs and were therefore selected. Fabric exposes a documented catalogue. Forge required additional investigation because the needed version information is obtained from its Maven metadata rather than from a dedicated public REST API.

Hoppscotch was used during this phase to inspect HTTP responses and understand provider data. Some orchestration behavior, particularly complete migration of a server and its mods, was postponed until a user interface existed. Exercising a long stateful workflow through isolated HTTP and service calls would have made diagnosis unnecessarily difficult. The interfaces and the basic operations were implemented first, then the postponed behavior was completed when it could be driven and observed through the application.

3.3.3 User-interface implementation

The user interface was developed during June 2026. The first version was intentionally rudimentary. Its purpose was to expose every functional workflow and reveal whether the service contracts contained the information required by a real interaction. This phase confirmed that the back end and front end could not be developed as two strictly consecutive products. Several services evolved when the screens made the order of selections, validation needs and change summaries concrete.

By early July the functional requirements were accessible through the interface. The remainder of July was assigned to visual refinement, clearer feedback and edge-case testing. Form controls generated most of the unexpected work. Their MAUI behavior did not always match the assumptions made in the first reusable form implementation, so binding, selection and validation logic required revision. These corrections extended the final defect-removal phase by approximately one week into early August. This was the only material deviation from the high-level schedule.

3.4 Risk management

The planning process did not maintain a separate formal risk register, but several technical risks influenced the order of work. External provider changes were the main risk. The application cannot control the structure, availability or authentication rules of remote catalogues. Provider access was therefore placed behind ports and mapped into domain objects. A change to one supplier should remain localized to its adapter unless it changes the meaning of the domain information.

Framework uncertainty was another risk because the project began without prior MAUI experience. Unexpected limitations in binding, navigation or native control behavior could have forced extensive changes after the application services were already implemented. The early feasibility month and the initial architecture phase reduced this risk before most application behavior was built. The final form-control problems described in the previous section show that the risk did materialize to a limited extent, producing a one-week delay rather than requiring a change of framework.

Server migration presented a risk of leaving an instance unusable or losing installed components. A new Minecraft version may not have a compatible release of the existing mod loader or every installed mod, and a download can fail after earlier files have already been changed. To reduce this risk, the migration workflow evaluates dependent components, shows the expected changes before confirmation and reports mods that were removed, required, incompatible or could not be downloaded. Its implementation was postponed until the interface could expose those consequences clearly. The remaining risk is that file-system changes are not performed as one atomic transaction, so an interruption at the wrong point can still leave a partly modified directory. Chapters 4 and 5 describe this limitation and the proposed use of staged updates and backups.

3.5 Assessment of the process

Weekly milestones were a good fit for exploratory integration work. They provided a definition of progress that was more meaningful than time spent, and they allowed knowledge gained from one provider to improve the next integration. The approach also made it possible to change service contracts during interface development without invalidating a large plan of detailed tasks.

The process had two limitations. First, the 10 to 20 hour weekly range was an estimate rather than recorded time, so it cannot support an exact cost calculation. Second, the absence of a detailed historical backlog means that the final sequence can only be reported at phase and milestone level. For a future team project, a shared Kanban board and a Scrum-style iteration review would provide better coordination and a more auditable history. For this individual project, the combination of a Gantt overview, source annotations and milestone commits was sufficient to keep the implementation directed toward the functional scope.

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

4.1 Users and operating context

Wave Launcher is designed for a player who owns Minecraft: Java Edition and wants to host a private server from a Windows computer. The target user can select gameplay options, Minecraft versions or mods, but is not assumed to understand Java compatibility, command-line flags, directory layouts, dependency graphs or network address translation. This distinction determines where responsibility lies. The user decides what kind of server is wanted; the program converts that intent into files, processes and provider requests.

The application is local and single-user. It is not an administration service intended to remain active independently of its interface, and it does not provide accounts or remote multi-user control. A server process exists only while Wave Launcher is running. This behavior gives the user a simple mental model: closing the program also ends the locally managed sessions. Persistent configuration survives between launches, but execution state does not.

The main workflows are server creation, edition of an existing server, execution and console interaction, Java management, and application configuration. Mod management is part of server revision rather than a separate global library because compatibility depends on the selected Minecraft version and mod loader.

4.2 Quality requirements

The functional requirements in Section 1.4 state what the application must do. Several non-functional properties constrain how those operations are implemented.

Usability. Server management must be possible without editing a JSON file, a properties file or a command line. Technical failures must be converted into visible messages,

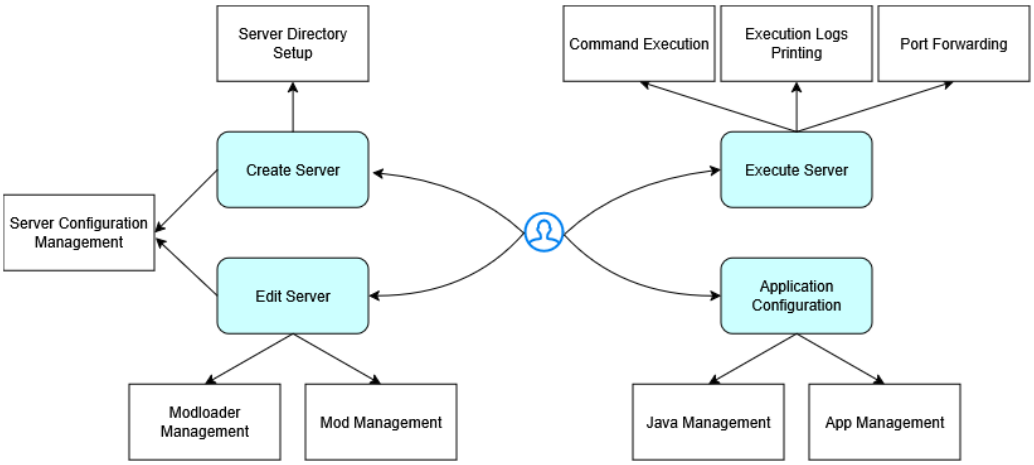

Figure 4.1: Principal user workflows in Wave Launcher.

and long operations must show that work is in progress. Related selections must be filtered so that the interface does not knowingly offer impossible combinations.

Compatibility. The completed product targets Windows. Provider and domain components should not depend on Windows when their work is portable, allowing another platform adapter to be added later. Minecraft, loader, mod and Java compatibility must be derived from provider metadata rather than encoded as a fixed list wherever such metadata exists.

Maintainability. External systems must be replaceable without rewriting the central use cases. File persistence, image conversion, server execution, Java acquisition, mod catalogues and port mapping are consequently accessed through application interfaces.

Recoverability. Persistent metadata must remain readable across normal application restarts. Failed downloads should not be recorded as successful installations. Migration must report components that cannot be retained. Since the current application modifies a real server directory rather than a transactional store, complete rollback of every file operation is not guaranteed; this constraint is relevant when interpreting the migration design.

Responsiveness. Network, installation and process operations use asynchronous interfaces so that the UI thread is not intentionally blocked while files are obtained or a catalogue is queried. The interface displays a busy overlay during operations that cannot complete immediately.

Security and privacy. Server worlds, configuration and Java runtimes remain on the local computer. CurseForge credentials are application configuration and must not be embedded in source code. Port mapping is requested only when a server is started and is released when it stops or startup fails.

4.3 Architectural design

4.3.1 Choice of architectural style

Wave Launcher follows a hexagonal architecture. At the centre are domain objects describing servers, Java installations, Minecraft versions, mod loaders, mods and device information. The application project surrounds this model with ports. Input ports express the operations available to the interface, while output ports describe what the application needs from files, remote providers and the operating system. The infrastructure project supplies the corresponding adapters and orchestration services.

This structure is useful because Wave Launcher depends on many replaceable external mechanisms. A Modrinth response and a CurseForge response have different schemas, but neither schema needs to leak into the server editor. Both adapters map their data into the same domain types. The same applies to Mojang and Adoptium Java packages, Forge and Fabric loaders, and JSON repositories. The design does not remove provider differences, such as the CurseForge token requirement, but it gives those differences a defined location.

Infrastructure contains both adapters and implementations of application services. The object graph is assembled manually by the `AppComposition` composition root instead of by a dependency-injection container. I evaluated dependency injection during development, but its integration with the MAUI interface made the creation of UI components disproportionately complex for this project. The extra indirection also made new screens harder to follow without giving a clear benefit at the current scale. Manual composition was therefore chosen as the simpler option. It produces a long initialization method, but every adapter, service and view-model dependency can be traced from one place. Section 5.1 shows how this decision appears in the code.

4.3.2 Solution projects

The solution is divided into four projects.

Wave.Domain contains records, entities, enumerations and state shared by the use cases. It has no reference to the interface or infrastructure projects.

Wave.Application defines input, intermediate and output contracts. It references the domain because its operations exchange domain objects.

Wave.Infrastructure implements repositories, external catalogues, installers, process execution, image transformation, port mapping and the services that coordinate them.

Wave.Ui provides the .NET MAUI application, XAML pages, reusable controls and view models. It is also the composition root that connects concrete adapters to their interfaces.

Dependencies point inward toward the domain and application contracts. The domain does not know that persistence uses JSON or that the graphical layer uses MAUI. Infrastructure can be replaced one adapter at a time as long as the corresponding contract remains meaningful.

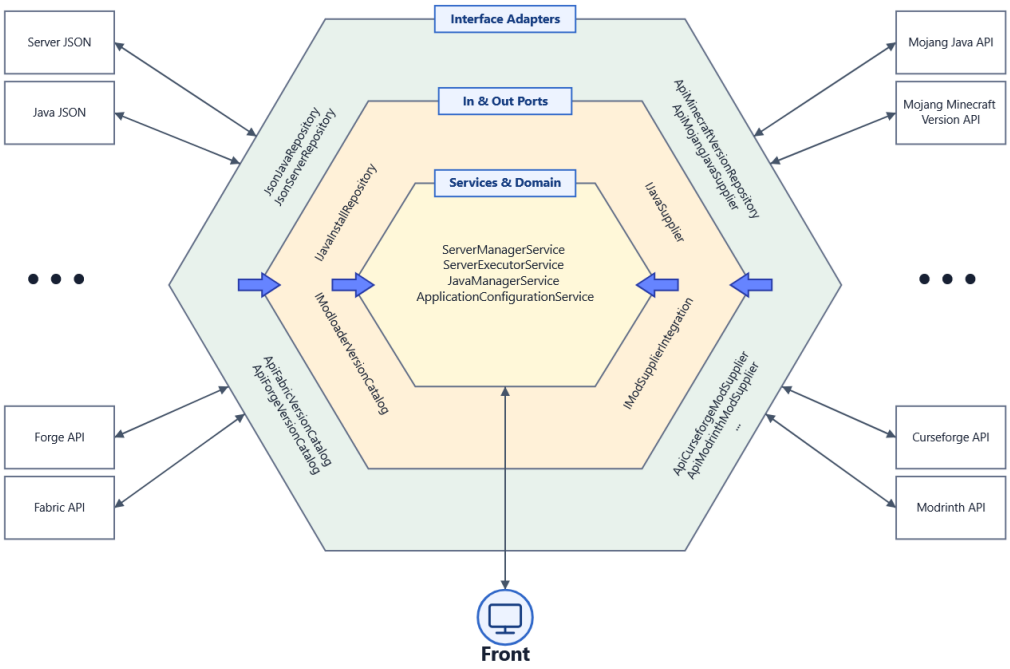

Figure 4.2: Hexagonal architecture and dependency direction.

4.4 Domain model

4.4.1 Server aggregate

The `Server` class is the central persistent object. It has a generated identifier, a display name, optional icon data, execution flags, EULA state and a dictionary of server properties. It refers to a `MinecraftVersionInstallation`, an optional `ModloaderInstallation`, a collection of `ModFile` objects and a selected `JavaInstallation`. The model stores the state required to reopen the server editor without rediscovering every choice from the file system.

Runtime state is deliberately excluded from this aggregate. A running Java process is represented by `IServerSession` and held by `IServerExecutorService`. This prevents process handles and observable output streams from being serialized. It also makes a clear distinction between a `Server` definition, which is persistent, and a server session, which exists only in memory.

`ServerQuery` is used as an editable transfer model for creation and modification. It can contain catalogue-level choices such as `MinecraftVersionInfo` before they have become installed artifacts. Comparing a query with the stored server produces `ServerChanges`, which lets the interface show the consequences of modifying or updating a server.

4.4.2 Version and installation types

Catalogue information and installed information are separate types. A `MinecraftVersionInfo` catalogue item contains an identifier, type and details URL. Retrieving `MinecraftVersionDetails` supplies the server download and required Java version. Once installed, the server holds a `MinecraftVersionInstallation`. A similar distinction exists among `JavaVersion`, `JavaPackage` and `JavaInstallation`, and among `ModloaderInfo`, `ModloaderPackage` and `ModloaderInstallation`.

This separation prevents a remote URL from being mistaken for a local artifact. It is particularly important for Java, where a supplier can expose compressed archives or a manifest made of many files. The installer chosen for a package creates a normalized `JavaInstallation` with an executable path that server execution can consume without knowing how the runtime was delivered.

4.4.3 Mod representation

Mods require more metadata than a simple file path. `ModInfo` identifies a project and its supplier; `ModVersion` describes a compatible release; and `ModFile` combines project and version information for installation. A mod file contains artifacts and dependencies. Dependencies are classified so that required and incompatible relations can influence installation.

Supplier identity remains in the domain model through `ModSupplierType` because identifiers from Modrinth and CurseForge are not interchangeable. This is an intentional provider distinction, unlike the HTTP response types, which remain inside their adapters. `PaginationState` is also represented independently so the same UI component can browse catalogues without depending on one provider's pagination fields.

4.5 Ports and adapters

4.5.1 Input ports

Input ports group operations by user intention. `IServerManagerService` creates, edits, deletes and retrieves server definitions. `IServerExecutorService` starts and stops sessions and submits commands. `IMinecraftCatalogService`, `IModloaderCatalogService` and `IModCatalogService` retrieve Minecraft versions, mod loaders and mods respectively. `IJavaManagerService` lists suppliers and installations and performs installation or removal. Application configuration and device information are exposed through `IApplicationConfigurationService` and `IDeviceInformationService`.

Separating management from execution avoids making the server entity responsible for process control. It also lets the servers page query execution state independently of the saved configuration. The interface works through these contracts, which keeps view models from reading JSON files or starting `java.exe` directly.

4.5.2 Intermediate services

Some operations are meaningful only as part of server management but still deserve isolated responsibilities. `IVersionManagerService` downloads the selected Minecraft

server artifact. IPropertiesManagerService and IEulaManagerService translate domain state to the text files consumed by Minecraft. IModloaderManagerService and IModManagerService install and migrate their respective components. IServerPathResolver gives all of these services a consistent directory layout.

Together, these services make it possible to treat creation and editing as coordinated workflows rather than as unrelated file operations. Creation has to respect the dependencies between the server directory, Minecraft version, native configuration files, optional loader, mods and persisted metadata. Editing first identifies what changed and then invokes the managers affected by that change. The exact order and the corresponding service calls are discussed in the implementation chapter.

4.5.3 Output ports

Output ports cover persistence, providers and operating-system facilities. IServerRepository, IJavaInstallRepository and IApplicationConfigurationRepository store servers, Java installations and application configuration. IMinecraftVersionRepository and IModloaderVersionCatalog obtain Minecraft and loader versions. IModSupplierIntegration and IJavaSupplier represent remote mod and Java providers. IServerExecutor executes servers, IImageTransformer transforms images, IDeviceInformationRepository reads device information, and IPortMapper creates network mappings.

Each adapter owns the translation between an external representation and the domain. ApiMinecraftVersionRepository deserializes Mojang's version manifest and maps its DTOs to version information. ApiForgeVersionCatalog parses Forge's Maven metadata as XML. Fabric, Modrinth, CurseForge and Adoptium responses use provider-specific DTOs within their respective adapters. JsonServerRepository, JsonJavaRepository and JsonApplicationConfigurationRepository serialize persistent domain state, while WindowsServerExecutor wraps System.Diagnostics.Process.

4.6 Persistence and directory design

JSON persistence was chosen because the data volume and relationships are small. The application needs to retain application settings, the list of managed Java installations and the configuration of each server. A relational database would add deployment and mapping complexity without providing a material benefit for the current access patterns. Server worlds and executable artifacts already live as files, so the JSON documents are metadata rather than the only copy of the installation.

Repository interfaces keep that decision reversible. If later versions need concurrent access, querying over many users or stronger transactions, a database adapter can implement the same repository roles. Some contract changes might still be needed for efficient querying, but UI and provider code would not have to know the storage technology.

ServerPathResolver centralizes the application directory, server collection, Java collection and temporary workspace. Each Server receives its own root directory. Files such as server.properties, eula.txt, the vanilla server JAR, the normalized loader JAR and the mods directory are derived from that root. Central path construction reduces the risk that different managers disagree about where an artifact belongs.

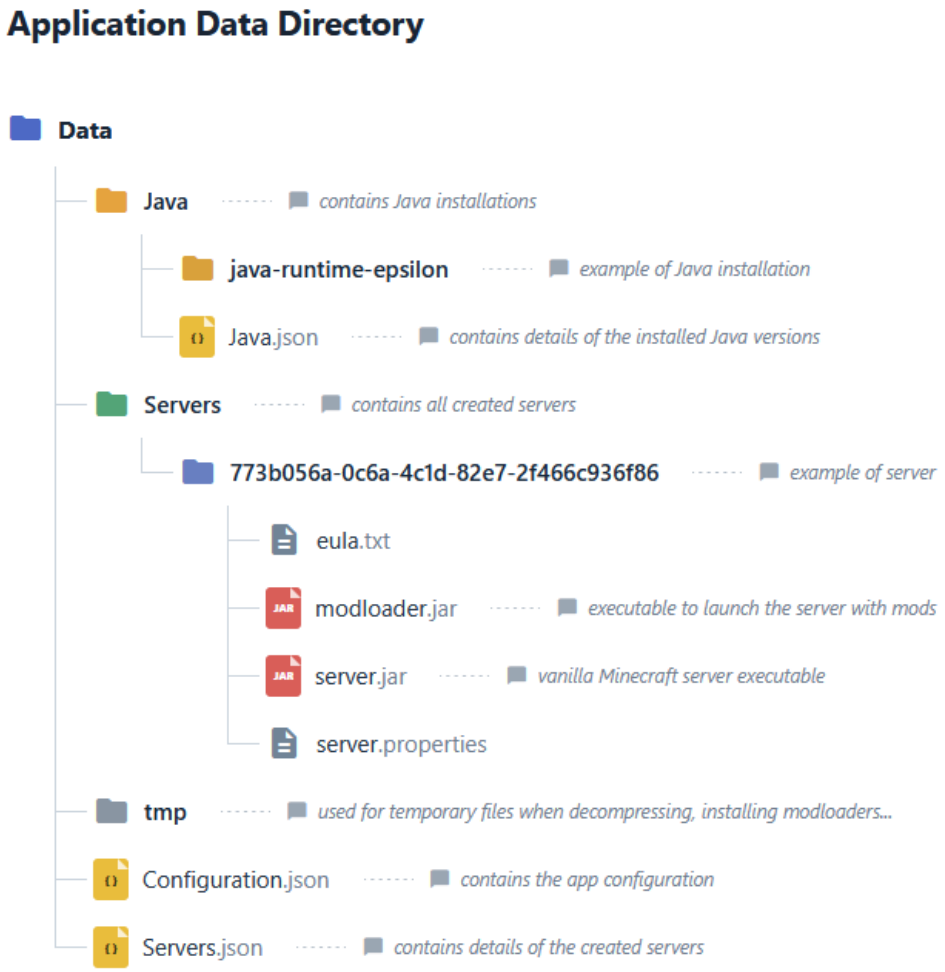

Figure 4.3: Application data and server directory layout.

4.7 Design of the principal workflows

This section describes the responsibility and order of each workflow. It intentionally stays at design level. Concrete methods, control flow and selected code excerpts are presented in Chapter 5.

4.7.1 Server creation and editing

The server editor collects the chosen Minecraft release, Java runtime, properties, icon, execution flags and EULA value in an editable query. Required information is validated before the query reaches the application service. The creation workflow then constructs a persistent server definition and performs the necessary file operations in dependency order.

Editing uses the same conceptual form but begins from the stored aggregate. At this stage, the mod-loader and mod-management controls become available because the initial server setup has already been completed and the application has a concrete

Minecraft version and directory against which compatibility and installation operations can be performed. `ServerManagerService` compares installed state with the submitted `ServerQuery` and coordinates both the requested edit and any additional changes needed to preserve compatibility. A Minecraft version change downloads the new server software and reevaluates the installed loader and mods against the selected release. Changing the mod-loader type also causes the mod set to be reevaluated because mods built for one loader cannot be retained under another without a compatible replacement. Even when the Minecraft version and loader remain unchanged, adding or replacing a mod can produce further changes: `ModManagerService` follows its required dependencies, adds compatible dependency files automatically and rejects declared incompatibilities.

The edit workflow returns its automatic consequences as a change summary. This summary identifies a loader that had to be removed and separates mods into deleted, failed, incompatible and automatically required collections. When no additional compatibility action is needed, there is no summary to display. Otherwise, the interface uses it to explain why the final installation may differ from the choices made directly by the user. The resulting metadata is saved only after the management workflow has applied both the requested and automatic changes.

The design favors a single coherent edit over a set of isolated buttons that can silently produce incompatible state. The change-summary popup makes potentially destructive consequences visible before confirmation.

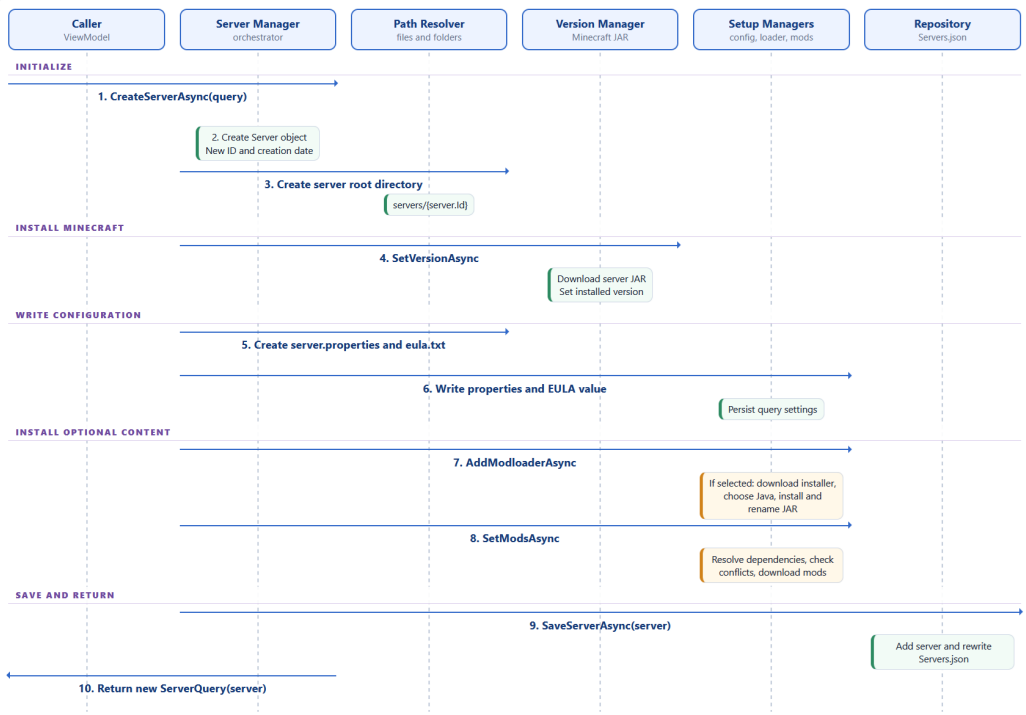

Figure 4.4: Sequence for creating a server instance.

4.7.2 Java management

Java management starts from the device operating system and architecture, which restrict the supplier results to usable packages. The two supported suppliers use different delivery formats: Adoptium provides compressed releases, whereas Mojang provides runtime manifests. The design represents both as Java packages, chooses an installer according to the package type and normalizes the result as a managed Java installation.

Only managed installations are presented as Wave installations. This avoids changing a system-wide Java configuration and lets multiple major Java versions coexist. A runtime cannot be removed when a saved server depends on it. During server execution, the selected installation supplies the exact executable path rather than relying on the system `PATH`.

4.7.3 Mod-loader management

The mod-loader workflow selects a catalogue according to the requested loader type. Forge derives its versions from Maven metadata, while Fabric exposes public catalogue endpoints. Despite this difference, both produce the same domain-level package and installation concepts. Packages are downloaded to temporary storage, installed with a managed Java runtime and normalized to a predictable loader path.

A server can have at most one loader. Changing the Minecraft version asks the catalogue for a loader release compatible with the new game version. If none exists, the loader is removed and the result is reflected in the server state. Removing a loader also removes associated mods because they cannot be assumed to run without that environment.

4.7.4 Mod discovery and dependency resolution

`ModsPopupViewModel` sends the selected Minecraft version, loader and `PaginationState` to an `IModSupplierIntegration`. Results are therefore constrained before the user chooses a release. `ApiCurseforgeModSupplier` requires a token supplied through application settings; `ApiModrinthModSupplier` does not require the same credential.

When mods are added, `ModManagerService` recursively resolves required dependencies. A visiting set prevents cycles from causing unbounded recursion. Already installed or already planned projects are not downloaded twice. Incompatible relationships are checked in both directions because either the candidate or an installed mod can declare the conflict. Automatically added dependencies are returned in `ModMigrationResult` so that the interface can explain the final set.

Migration reevaluates every mod whose Minecraft version or loader differs from the changed server. The old artifact is removed, the supplier is asked for compatible releases and the newest returned version is selected. The result distinguishes deleted, failed, incompatible and automatically required mods. This information is more useful than a single success flag because a valid server migration can still mean that a particular mod is no longer available.

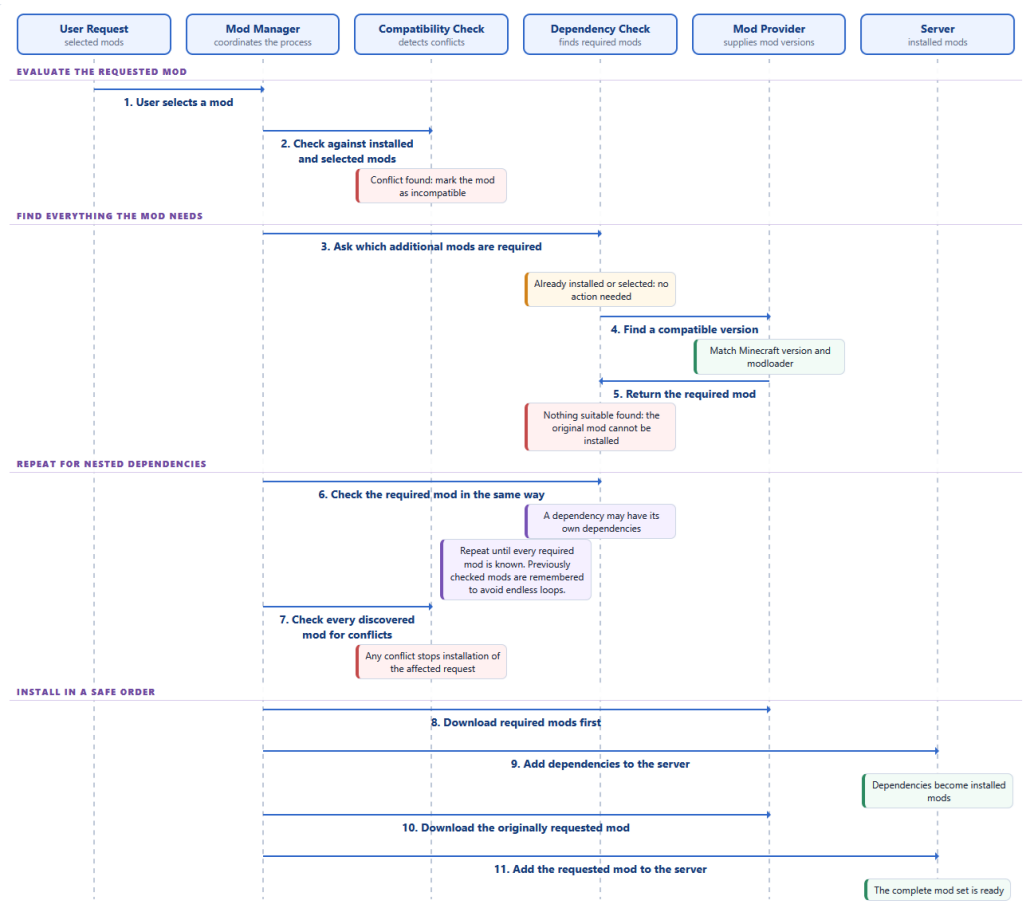

Figure 4.5: Recursive resolution of required and incompatible mods.

4.7.5 Execution and port mapping

Starting a server is a coordinated local and network operation. `ServerExecutorService` first rejects a second start of the same instance and checks whether another Wave session uses the requested port. It then requests a temporary mapping through `IPortMapper`. `SharpOpenNatPortMapper` attempts UPnP and NAT-PMP discovery and creates a TCP mapping with the same private and public port. If discovery or mapping fails, startup returns a specific `ServerStartResult` and the Java process is not launched.

After mapping succeeds, `ServerExecutorService` chooses the server or loader JAR, obtains the configured Java executable and asks `IServerExecutor` to start the process. Standard input, standard output and standard error are redirected through `IServerSession`. Output is exposed as an asynchronous sequence so `ExecutionViewModel` can append lines as they arrive. Commands written by the user go to standard input through `IServerSession.SendCommandAsync`.

The mapping is represented by `PortMappingLease` and owned by the private `RunningServer` record in `ServerExecutorService`. It is disposed when the server stops, when the process exits, or when process startup fails. This ownership is important because a mapping should not remain after the corresponding listener has gone away.

Router support is still an environmental requirement: automatic mapping depends on UPnP or NAT-PMP being enabled and available.

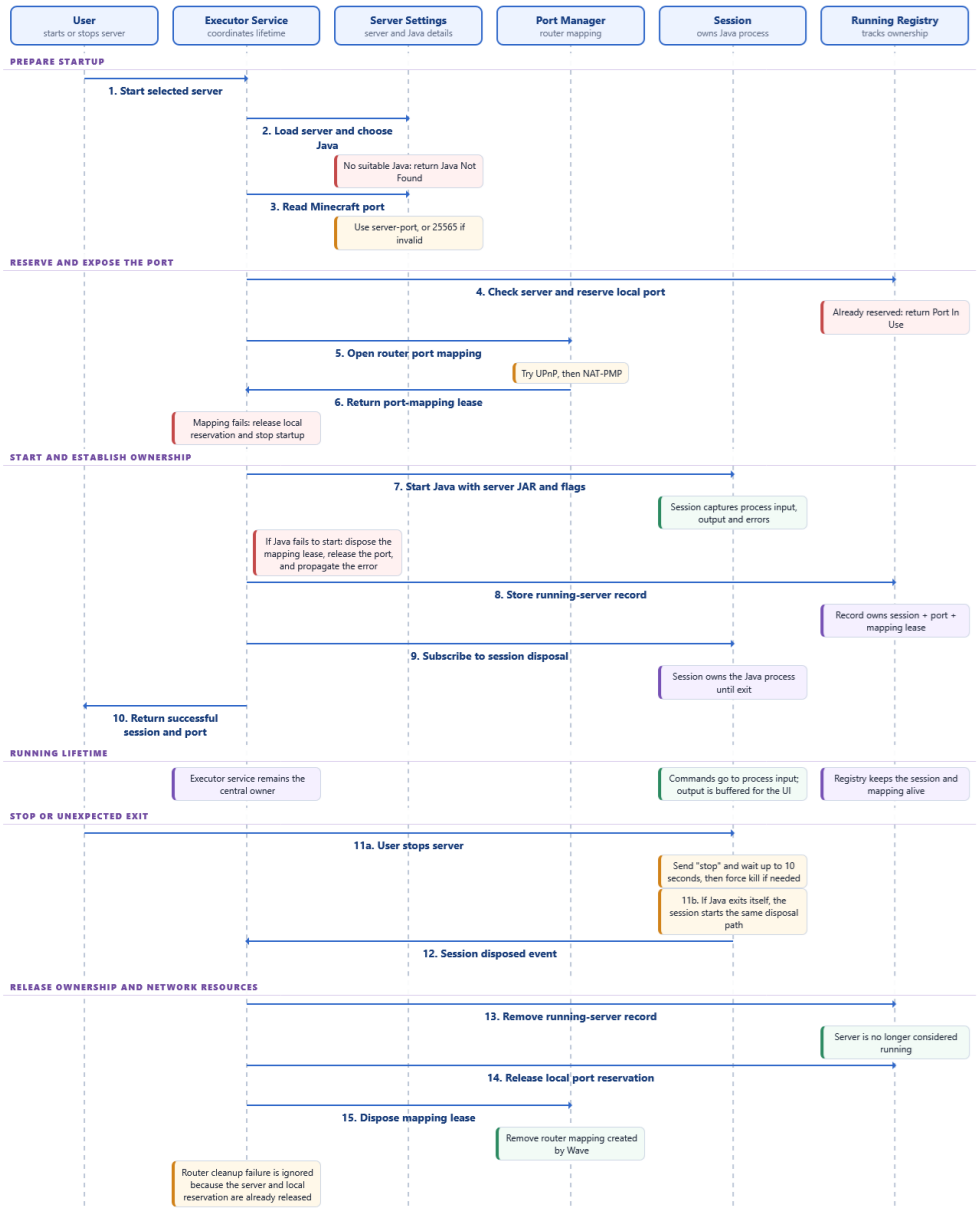

Figure 4.6: Server startup, process ownership and port-mapping lifetime.

4.8 User-interface design

The `Wave.Ui` project forms the presentation layer. It organizes the interface using XAML views and view-model classes: views define the visual structure, while view models expose screen state and invoke the application input ports. The project also contains `AppComposition`, which assembles the concrete infrastructure adapters and applicati-

on services before supplying them to the view models. This keeps construction at the outer edge of the system and prevents the presentation components from instantiating repositories, provider clients or process executors themselves.

The application shell exposes three main destinations: servers, execution and settings. The servers page is the starting point and displays persistent instances as cards. Creating or selecting a card opens the server editor. The editor divides a large configuration into general, properties and mods views. The execution page concentrates transient process interaction, while settings contains Java and provider configuration.

View models use commands and observable properties from CommunityToolkit.Mvvm. The XAML views bind to those properties and do not call repositories or external providers. Custom form components standardize labels, validation and submission. A paginated collection component is reused for provider results, and a busy overlay prevents repeated submissions during long operations.

The visual design uses a dark palette and card-based grouping familiar to game launchers. Provider logos help distinguish Mojang, Adoptium, Forge and Fabric choices. Destructive actions are separated from ordinary editing, and a changes popup is used when an update may remove or replace dependent components.

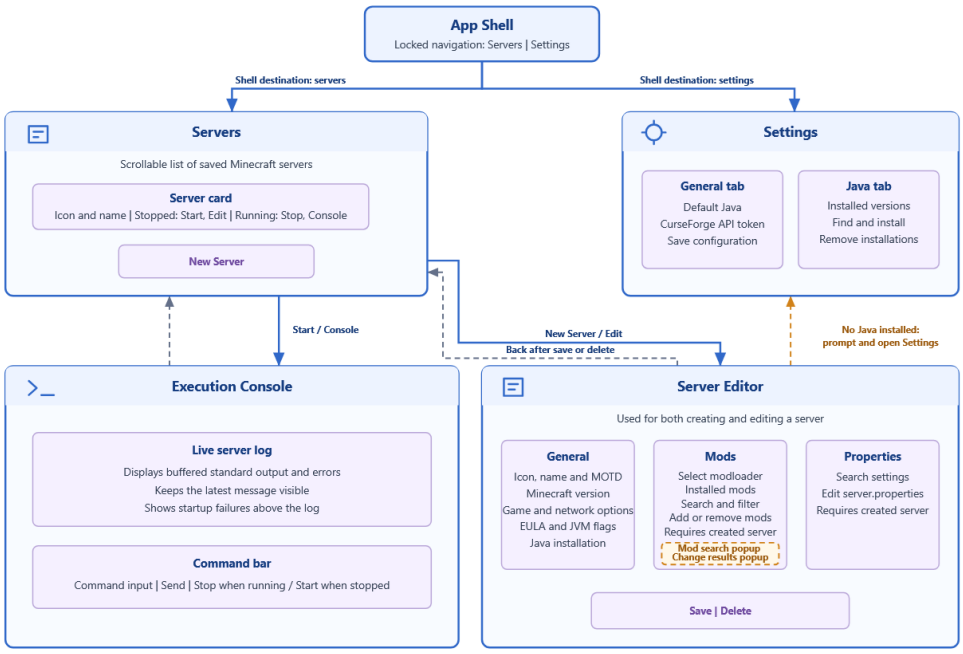

Figure 4.7: Navigation structure and principal application screens.

4.9 Technology selection

4.9.1 C#, .NET and MAUI

C# and .NET were selected both as a learning objective and for their suitability for a desktop application that performs HTTP, file-system and process operations. Asynchro-

nous tasks, JSON serialization, process APIs and a mature package ecosystem cover the core technical needs without introducing a second server-side runtime.

.NET MAUI supplies a shared XAML and C# UI project for Windows, Android, iOS and macOS. The present release is Windows-only because its executor and device-information adapters are Windows-specific and the project was validated on that platform. The framework choice nevertheless prevents the complete interface from being tied to a Windows-only widget toolkit. Supporting another desktop platform would require platform adapters and testing rather than a new application core.

A local web interface was the main alternative considered. It would have made browser technology available, but it creates a less direct lifecycle for the target user: launching an executable that opens a browser tab does not clearly communicate whether the underlying local service is still running. A desktop window provides an immediate indication that Wave Launcher and its server sessions are active. For a casual user installing a local tool, this model is closer to the expected behavior of an application.

4.9.2 Libraries

CommunityToolkit.Mvvm supplies observable properties and commands, while CommunityToolkit.Maui provides additional interface support. `System.Reactive` is used for process output. ImageSharp normalizes selected server icons. Markdig converts provider descriptions written in Markdown. SharpOpenNat provides UPnP and NAT-PMP discovery and mapping. HTTP clients and XML or JSON serializers implement provider communication.

These libraries are used behind project-level abstractions where their behavior affects the application core. SharpOpenNat, for example, is hidden by `IPortMapper`, and ImageSharp by `IImageTransformer`. UI-oriented libraries remain in the MAUI project because replacing the UI framework would already require new views and view models.

4.9.3 Development environment

Development was performed in Visual Studio Code with C# Dev Kit, .NET MAUI tooling and NuGet Gallery. Todo Tree supported daily tracking. Git and GitHub supplied local history and remote backup, with major milestones used as commit boundaries. Hoppscotch assisted during the external API phase. Artificial-intelligence tools were used mainly while diagnosing programming defects. No diagramming application formed part of the development process; the architectural illustrations in this report document the resulting system rather than reproducing design artifacts created earlier.

4.10 Design limitations

The architecture reduces coupling but does not make external services reliable. Catalogue operations require internet access, CurseForge requires a user-provided API token, and automatic network exposure requires a compatible router configuration. The application must report these conditions but cannot remove them.

Persistence is simple and appropriate to a local single-user application, but JSON files and server directories do not provide a shared transaction. A failure at an unfortu-

nate point in a multi-file migration may require recovery work beyond restoring one serialized object. Backups and staged directory replacement would strengthen a later release.

CHAPTER 5

IMPLEMENTATION

5.1 Application composition

The architectural chapter introduced the decision to use manual composition. In the implementation, this responsibility belongs to the static `AppComposition` class. Its initialization method first creates the application directories, then the JSON repositories, HTTP catalogues, installers, Windows process executor, SharpOpenNat mapper and image transformer. It uses these adapters to construct the intermediate managers, followed by the input services. Finally, small factory methods return fully initialized view models to the pages.

This is one of the less elegant parts of the code because the construction section is sizeable and the complete graph has a static lifetime. For a single local application instance, I found that trade-off acceptable. The benefit during development was practical: when a service required a new dependency, the complete construction path was visible in one file. If the application grows, smaller composition modules could make tests easier to set up without changing the architectural boundaries.

The following shortened excerpt from `AppComposition` illustrates why the selected approach remains easy to follow. Each service is constructed from previously created ports, and its dependencies can be identified without consulting registration rules or framework conventions.

```
serverPathResolver = new ServerPathResolver(
    appDirectory, serversDirectory, tmpDirectory);
propertiesManagerService = new PropertiesManagerService(
    serverPathResolver, serverPropertiesRepository);
eulaManagerService = new EulaManagerService(
    serverPathResolver, serverEulaRepository);
versionManagerService = new VersionManagerService(
    serverPathResolver, minecraftVersionRepository);
```

The excerpt also shows the dependency direction in practice. `PropertiesManagerService` receives an `IServerPropertiesRepository`; it never creates a JSON or text repository

internally. The same rule is followed by the EULA and version managers. As a result, the implementation can replace one adapter without changing the manager that uses its contract.

At startup, four logical storage locations are derived from MAUI's application-data directory: the application root, `Servers`, `Java` and `tmp`. Provider adapters use the temporary location for downloaded packages, while repositories and the path resolver use the persistent locations. The program does not assume that its installation folder is writable.

5.2 Server persistence and file management

5.2.1 JSON repositories

`JsonServerRepository`, `JsonJavaRepository` and `JsonApplicationConfigurationRepository` implement the persistent stores. Their interfaces make save, retrieval and removal explicit while shielding callers from filenames and serializer settings. A server's identifier is stable and is used both to distinguish records and to derive its managed directory.

Application configuration includes the default Java selection and the CurseForge API token. The configuration service trims the token and propagates it to CurseForge supplier instances. Keeping this operation in a service ensures that a token loaded at startup and a token changed through the settings view have the same effect.

JSON was not used for Minecraft's own settings. The server expects `server.properties` and `eula.txt`, so specialized repositories write those native formats. This distinction avoids treating every file as an application database record. Wave metadata describes the managed aggregate; Minecraft files remain in the form consumed by Minecraft.

5.2.2 Paths and cleanup

`ServerPathResolver` supplies paths for the root, vanilla JAR, loader JAR, properties, EULA and mods. Services request paths instead of reconstructing names. Creation ensures required directories exist, while deletion removes the complete server root only for the selected managed instance.

Temporary installer files are removed after successful use. Mod downloads are also guarded by cleanup: if a supplier throws during a download, artifacts known for that mod are removed when possible and the operation is reported as failed. Cleanup exceptions are logged separately so they do not hide the original failure.

5.3 Minecraft catalogue and installation

`ApiMinecraftVersionRepository` reads Mojang's global version manifest. Each entry is mapped to a domain item that identifies releases, snapshots and pre-releases and retains the URL for detailed metadata. When a user selects a version, the repository follows that URL to obtain the server download and Java information, then streams the JAR to the destination chosen by the path resolver.

The catalogue service filters snapshots unless the query enables them. This preserves the full provider catalogue without burdening the usual creation workflow with unstable releases. Property definitions are held by an in-memory repository because

they are part of the interface model rather than remote changing data. Their keys correspond to Minecraft's `server.properties` format, and their types determine which form component is created.

`VersionManagerService` distinguishes a catalogue choice from the already installed selection. When the versions differ, it retrieves details and replaces the managed vanilla artifact. This service is called by server creation and by editing, allowing both workflows to use the same installation behavior.

5.4 Java runtime management

5.4.1 Supplier adapters

Java support has two supplier adapters. The Adoptium adapter queries available feature releases for the device operating system and architecture, maps returned binary metadata and downloads a compressed package. The Mojang adapter obtains its runtime manifest and downloads the set of files described by the selected release.

These delivery formats are intentionally different in the implementation. `CompressedJavaPackage` is handled by `CompressedJavaInstaller`, which extracts ZIP or TAR-based archives, locates the Java executable and places the installation beneath the managed Java directory. `ManifestJavaPackage` is handled by `ManifestJavaInstaller`, which reconstructs Mojang's file tree from manifest entries and assigns executable information. The abandoned MSI path is not part of the active project. Adoptium is installed only from compressed packages, avoiding system-wide installer behavior and administrative side effects.

5.4.2 Selection and removal

The Java settings view obtains device information from `WindowsDeviceInformationRepository` and uses it to restrict supplier queries. Installed runtimes are read from the Java repository. After installation, the normalized domain object records supplier, version, artifact type, executable and removal location.

Before uninstalling a runtime, the Java manager checks the saved servers. Removal is refused if any server refers to that installation, preventing a configuration from retaining a dead executable path. A different runtime can first be assigned through the server editor. This rule is implemented in the use-case service rather than only in the interface, so it also protects future callers.

5.5 Forge and Fabric integration

Forge publishes version bundles through Maven metadata. `ApiForgeVersionCatalog` downloads and deserializes the XML document, maps each combined Minecraft and Forge version, filters it for the requested Minecraft release and reverses the result so recent releases appear first. The selected installer JAR is downloaded from the corresponding Maven path.

Fabric supplies loader and installer information through JSON endpoints. Its adapter combines the selected loader with a current installer package. Both adapters im-

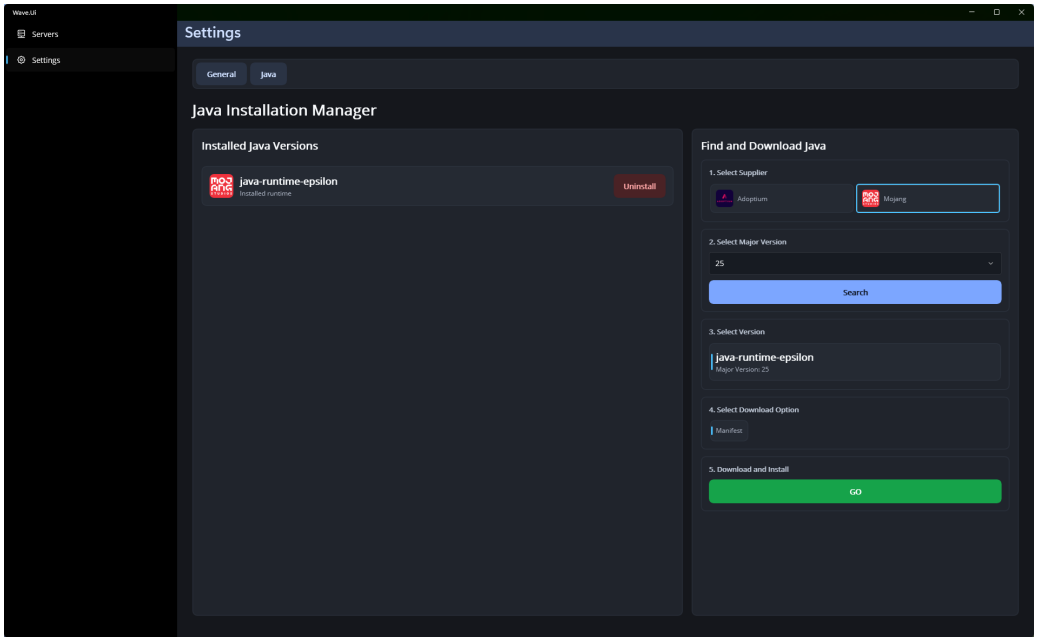

Figure 5.1: Java catalogue, package and installation screens.

plement `IModloaderVersionCatalog`; callers select them by `ModloaderType` and do not handle XML or Fabric DTOs.

Installation invokes the downloaded JAR with a managed Java executable in server-install mode. When the installer completes successfully, the resulting launcher JAR is located and moved to the normalized loader path. The stored `ModloaderInstallation` retains type, Minecraft version and loader version. Temporary installer material is then cleared.

Loader migration begins only when the server already has a loader. The catalogue is queried using the new Minecraft version. If there is no corresponding release, loader state is removed. Otherwise the old loader is removed, the first compatible catalogue result is installed and mod migration can continue against the new combination.

5.6 Mod provider integration

5.6.1 Common supplier contract

`IModSupplierIntegration` covers search, project details, compatible versions, download and token configuration. A supplier declares which `ModSupplierType` it handles. `ModCatalogService` selects the appropriate adapter, checks whether its configuration permits use and returns provider-independent domain data to the interface.

Search is paginated. The UI query includes Minecraft version and loader, which adapters translate into the filters required by their provider. Results expose a project name, summary, icon and provider identifier. Selecting a project triggers separate detail and version requests so the initial results remain compact.

5.6.2 Modrinth

The Modrinth adapter uses its project search and version operations without requiring a user API token. DTO mappers translate project type, authors, descriptions, files and dependency relations. File downloads are streamed into the server's mods directory. Markdown descriptions are converted for presentation in the mod popup.

5.6.3 CurseForge

CurseForge requests include an API token stored through the settings page. Operations fail with a clear configuration exception when the token is empty. The adapter maps CurseForge project and file identifiers into the same domain model used by Modrinth, while retaining supplier identity so later detail and download calls return to the correct platform.

The provider uses dependency classifications that do not map perfectly to the subset required by the application. Required and incompatible relations participate in automatic resolution. Unsupported relation types remain an adapter concern and are not silently treated as required dependencies.

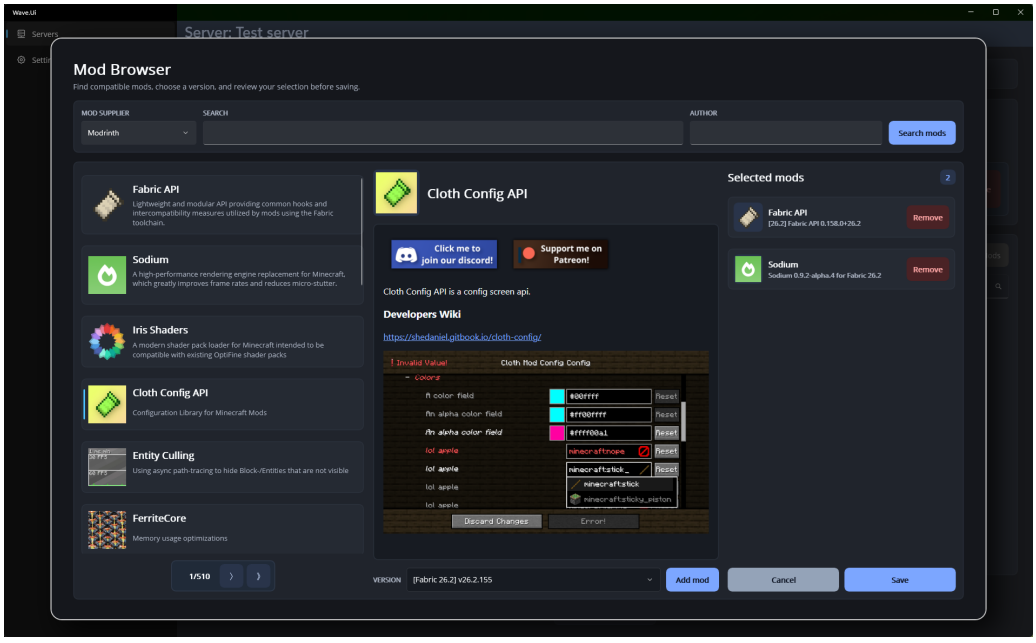

Figure 5.2: Mod search, details and selected-mod views.

5.7 Dependency installation and migration

`ModManagerService` compares requested and installed mods using the pair of project and version identifiers. Added items pass through recursive dependency resolution before download. Removed items have each known artifact deleted and are removed from server metadata.

Required dependency resolution receives the candidate, current server state, explicitly known selections, an accumulating dependency list and a visiting set. It first tests incompatibilities against installed and planned items. It then follows every required relation that is not already satisfied. If the dependency was explicitly selected, that exact selection can be reused. Otherwise the supplier is queried for a compatible version and its project information. Recursion processes the dependency's own requirements before adding it to the ordered download list.

The visiting set handles circular declarations. Encountering the same project on the current recursive path returns success rather than descending again. A project is added only once because installed and planned lists are checked at each step. These two guards protect against both cycles and repeated dependencies shared by several selected mods.

The central guard and recursive call in `ResolveDependenciesAsync` make this behavior explicit. The visiting set terminates a cycle, while any non-successful result from a nested dependency is propagated to the original installation request.

```
if (!visiting.Add(mod.ModId))
    return DependencyResolution.Success;

var result = await ResolveDependenciesAsync(
    dependencyMod,
    server,
    knownMods,
    dependencyMods,
    visiting);
if (result != DependencyResolution.Success)
    return result;
```

Downloads occur after resolution for each candidate. Automatically obtained dependencies are installed before the requesting mod. The result returns automatically required items as well as failures and incompatibilities. This lets the popup explain why the final set differs from the user's direct selection.

Migration uses the new Minecraft version and loader as its compatibility query. Each outdated mod artifact is uninstalled and the supplier is asked for alternatives. No available release means the mod is recorded as deleted. A replacement is processed through the same dependency routine used for ordinary additions. The interface can therefore present one consolidated migration result rather than leaving obsolete files in the directory.

5.8 Server creation and update orchestration

`ServerManagerService` owns workflows that span several specialized managers. Creation initializes the server from the submitted query, creates the root, installs the Minecraft version, writes properties and EULA state, installs an optional loader, adds selected mods, transforms the icon and saves the final aggregate.

The implementation keeps this order visible instead of hiding it in a generic pipeline. The following shortened fragment from `CreateServerAsync` shows the main sequence. Each call delegates one concern, while the service remains responsible for coordinating the complete use case.

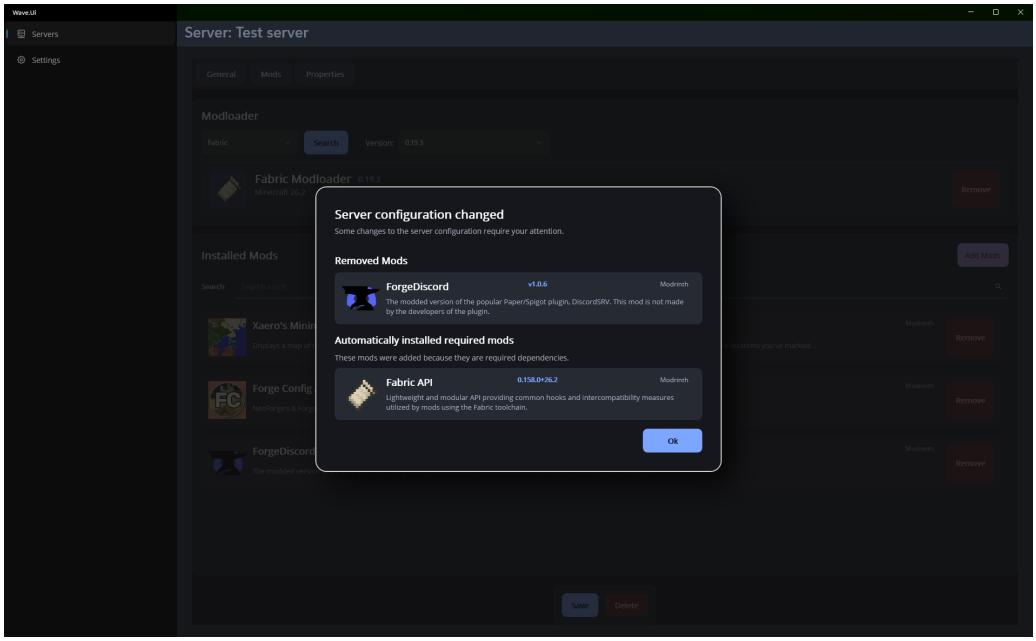

Figure 5.3: Change summary produced during a server-version migration.

```
serverPathResolver.CreateServerRootDirectory(server);
server = await versionManagerService.SetVersionAsync(server, query);

serverPathResolver.CreateServerPropertiesFile(server);
await propertiesManagerService.SetPropertiesAsync(server);

serverPathResolver.CreateEulaFile(server);
await eulaManagerService.SetEulaAsync(server, query);

await modloaderManagerService.AddModloaderAsync(server, query);
await modManagerService.SetModsAsync(server, query);
await serverRepository.SaveServerAsync(server);
```

This order matters. The version manager needs an existing root in which to place the server JAR, and the loader and mod managers need a concrete Minecraft installation against which compatibility can be checked. Persistence comes last so that a newly created record does not claim success before the main artifacts have been installed.

Update is more involved. It loads the existing aggregate, remembers the original Minecraft version and loader, and applies the submitted values. Version migration comes before loader and mod migration because their valid choices depend on Minecraft. If the loader type changes, the old loader is removed before the new one is installed. The method then either migrates the mods or applies an ordinary mod-set change. Failures, incompatibilities, deleted mods and automatically required dependencies are combined into `ServerChanges`. This object is returned only when there is something the user should be told about.

The service also supplies a preview of changes for the popup. A change object communicates version replacement and dependent modifications without making

XAML calculate domain consequences. This division helped during UI development because the preview and final operation could evolve together in the application layer.

Deletion removes the server metadata and its managed directory. The UI prevents ordinary editing while an operation is in progress, but execution and management are separate services. A production-hardening step would add an application-level rule that makes deletion of a running server impossible even for a caller outside the current UI.

5.9 Process execution and console

`WindowsServerExecutor` creates a process using the selected Java executable. Its working directory is the server root, its arguments include configured execution flags and the appropriate server or mod-loader JAR, and shell execution is disabled. Standard streams are redirected and no additional console window is created.

The returned `ServerSession` exposes output and lifecycle information. Standard output and error are observed by `ExecutionViewModel`, which maintains the log shown in the execution page. Commands are written to the process input. Stopping requests Minecraft's normal stop command before session cleanup, allowing the game server to save world state.

Log distribution is implemented inside `ServerSession`. Two asynchronous pumps read the process's standard-output and standard-error streams line by line. Each produced line is written into a circular array at `bufferIndex modulo bufferSize`; the index then advances and a task-completion signal wakes consumers waiting for new output. Access to the array, index and completion state is protected by a lock, so both pumps can publish without corrupting their shared state. The current buffer contains ten lines, which bounds its memory use and retains the most recent output when producers advance faster than a consumer.

`GetOutputAsync` exposes this data as an `IAsyncEnumerable<string>`. Every invocation maintains its own `previousIndex`, copies the unread portion of the circular buffer while holding the lock, releases the lock before yielding the lines, and then waits on the current production signal. Consumers therefore do not remove entries from a shared queue. Several independent consumers can follow the same session concurrently, each progressing at its own rate. Cancellation stops only the corresponding enumeration, while completion of both stream pumps wakes all remaining consumers so their enumerations can finish normally.

The important detail is that no value is yielded while the lock is held. The method first copies the currently unread lines into a local queue, then returns them to the consumer. The central part of that operation is shown below.

```
lock (bufferLock)
{
    currentIndex = bufferIndex;
    long unreadLines = currentIndex - previousIndex;
    long retainedLines = Math.Min(bufferSize, unreadLines);

    for (long i = currentIndex - retainedLines; i < currentIndex; i++)
        currentBuffer.Enqueue(buffer[i % bufferSize]);
```

```
    waitProduced = signal.Task;
    if (completed) shouldBreak = true;
}

foreach (string line in currentBuffer)
    yield return line;
```

The names in this excerpt have been shortened slightly for readability, but the control flow matches `ServerSession`. Copying before yielding avoids holding a synchronization primitive across consumer code, which could otherwise delay both output pumps.

The UI is currently one such consumer, but the design allows other output processors to subscribe without changing process execution. A future persistence component could consume the same stream and write the server log to a text file for later consultation. Another component could inspect reported events and trigger configured actions, such as moderating a player when prohibited language is reported or restarting a server after Minecraft reports that a game tick is taking too long. These extensions should parse known message formats and apply explicit policies rather than embedding them in `ServerSession`, whose responsibility remains output distribution.

The circular buffer is a live distribution mechanism, not a durable log. A newly attached consumer receives at most the retained lines, and a consumer that falls more than ten lines behind can lose overwritten output. Reliable file logging would consequently require the persistence consumer to remain active and keep pace, or a later revision to introduce a larger buffer, backpressure or a dedicated durable logging channel.

`ServerExecutorService` maintains an in-memory dictionary from server identifiers to running-session records. A record also contains its port and mapping lease. The dictionary prevents duplicate starts and lets the servers page display current execution state. Because it is deliberately transient, restarting Wave Launcher begins with no claimed sessions.

5.10 Automatic port mapping

The requested server port is read from the properties dictionary, with Minecraft's default used when the value is absent or invalid. The executor service checks its own running records for a conflict before making any network change. It then calls `IPortMapper.OpenAsync`.

`SharpOpenNatPortMapper` tries supported discovery protocols with a bounded timeout. For UPnP it can inspect a specific existing mapping. If an equivalent mapping already targets the local host and port, the adapter returns a lease without duplicating it. Otherwise it creates a TCP mapping whose public and private ports match. The returned `PortMappingLease` contains the cleanup action needed to remove a mapping created for the session.

If mapping fails, `ServerStartResult` reports the failure and process creation does not proceed. If process creation fails after mapping, the lease is disposed in the error path. Normal stop and observed process exit also dispose it. This sequence prevents a successful network side effect from being orphaned by a later local failure.

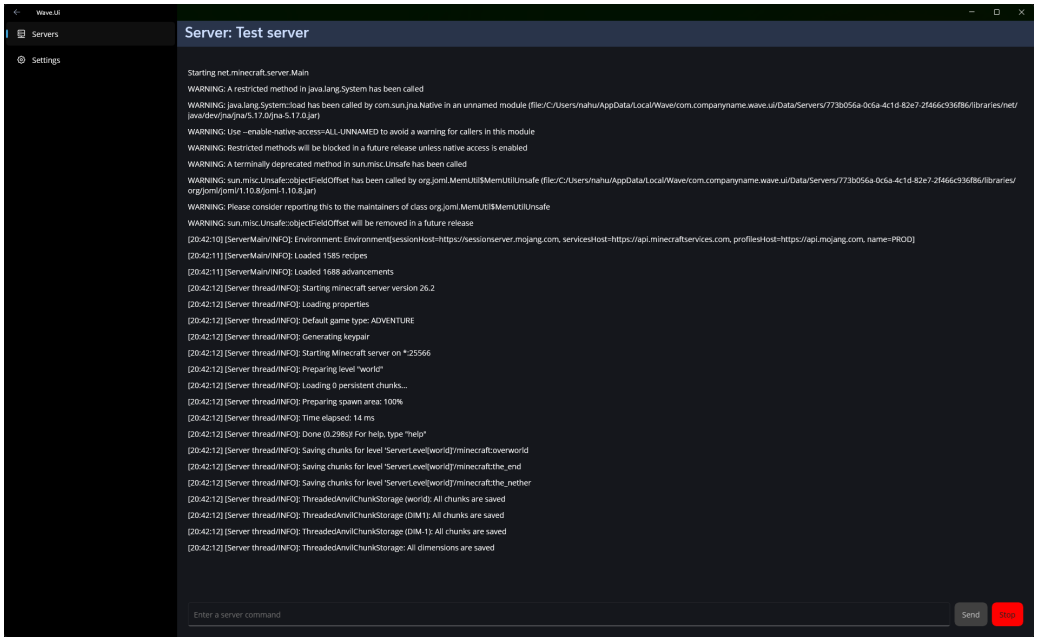

Figure 5.4: Execution page with redirected server output and command entry.

The lease itself contains only the mapped port and the operation required to release it. Its disposal guard is atomic, so cleanup can safely be requested by both explicit stopping and process-exit handling without removing the mapping twice.

```
public async ValueTask DisposeAsync()
{
    if (Interlocked.Exchange(ref disposed, 1) != 0) return;
    await releaseAsync();
}
```

Automatic mapping cannot guarantee external reachability. Some routers disable discovery, some networks place users behind carrier-grade NAT, and host firewalls can still reject traffic. The implementation removes the need for manual router configuration when the local environment supports UPnP or NAT-PMP, while failure feedback remains necessary for unsupported networks.

5.11 MAUI user interface

5.11.1 Shell and pages

`AppShell` defines navigation among servers, execution and settings. Each content area uses a page, a view model and smaller views. The server editor, for example, composes general, property and mod views instead of placing the complete form in one XAML file. This mirrors the functional grouping used in the domain and keeps provider browsing separate from basic server identity.

Server cards show identifying and state information. The selected server is passed to the editor through its view model, where a query model collects changes. The same

editor can represent a new or existing server, reducing divergence between creation and update validation.

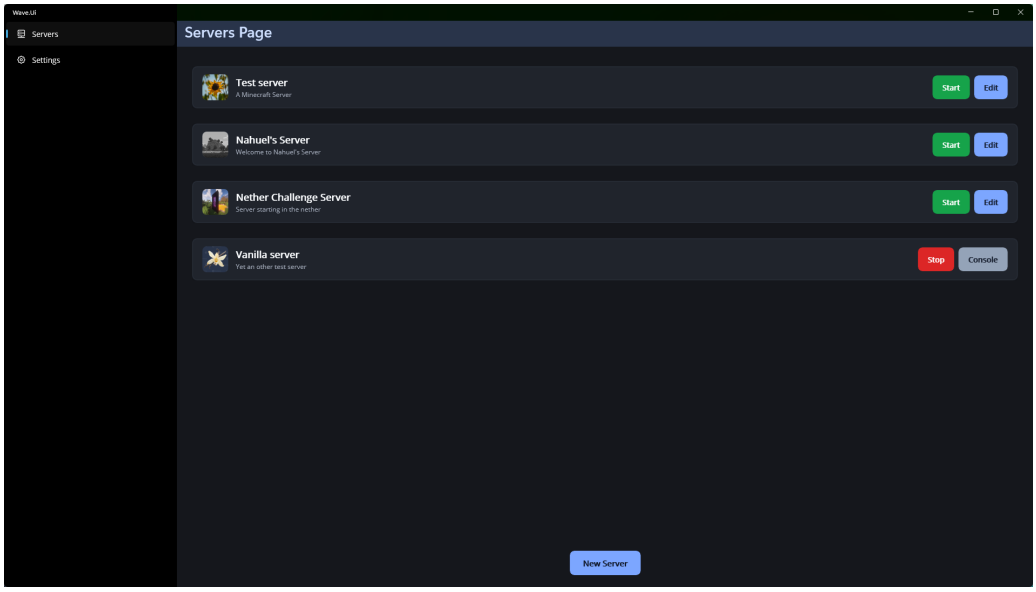

Figure 5.5: Server list page.

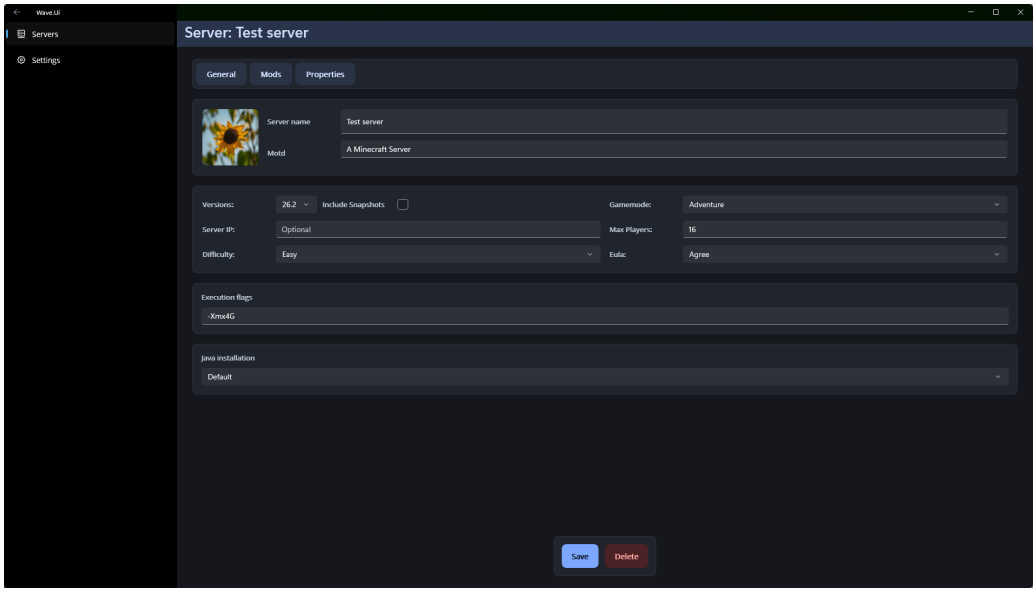

Figure 5.6: General server-configuration page.

5.11.2 Reusable forms

The form subsystem contains text entries, check boxes, pickers and a submit button that implement a common form-element contract. Reflection helpers read and write

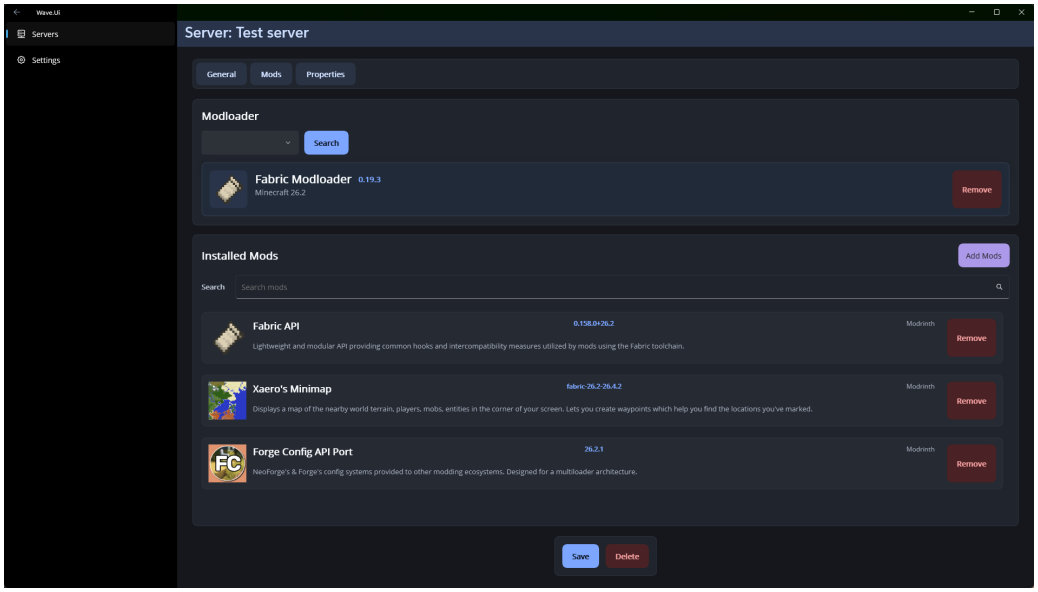

Figure 5.7: Installed-mods page for a server.

nested model paths. This allowed the large server-properties set to be generated from definitions instead of being manually bound one property at a time.

The `Form` control discovers elements that implement `IFormElement`, assigns the current model to them and asks each element to validate itself before submission. Nested forms are deliberately skipped so that a parent form does not validate or overwrite a child form's controls. The submission method is small because traversal and validation are kept as separate responsibilities.

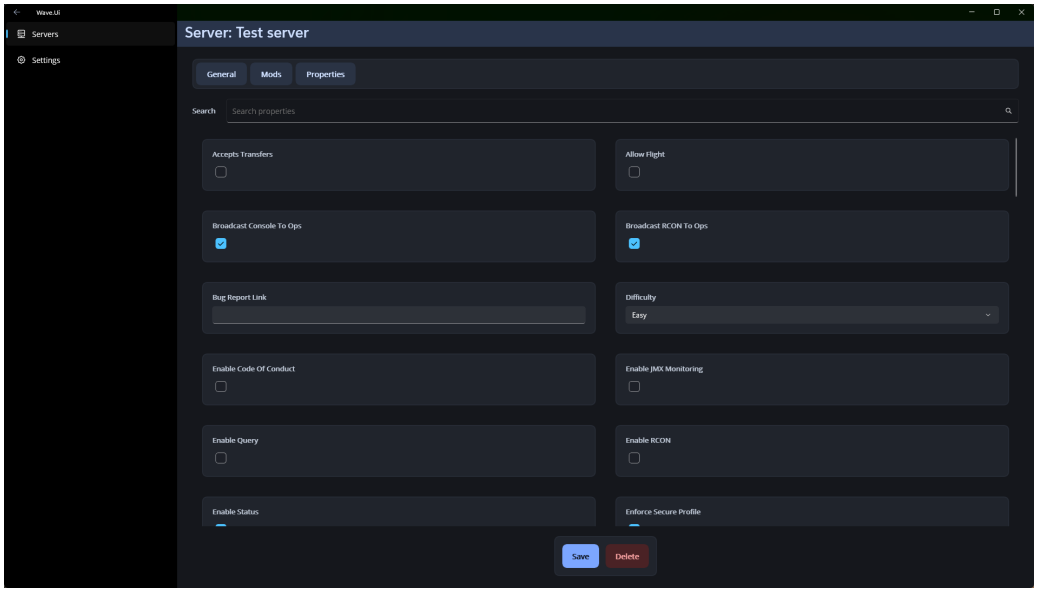

Figure 5.8: Server-properties configuration page.

```
public void Submit()
{
    if (!Validate())
        return;

    if (SubmitCommand?.CanExecute(Model) == true)
        SubmitCommand.Execute(Model);
}
```

Reusable forms also produced the largest schedule overrun. MAUI picker selection, generated bindings and changing binding contexts introduced cases that were not visible in simple controls. The implementation added guarded selection updates and explicit synchronization to prevent recursive changes. This work occupied the additional correction week noted in Chapter 3.

5.11.3 Feedback and pagination

Long operations set busy state and display `BusyOverlay`. The overlay communicates that the submitted action is still active and blocks accidental duplicate input. Popups present mod details and server change summaries without navigating away from the editor.

`PaginatedCollectionView` and `ObservablePaginationState` provide first, previous, next and last navigation where the supplier exposes sufficient totals. The component observes state changes and updates the displayed collection. Keeping it supplier-neutral lets Modrinth and CurseForge searches share interaction behavior despite different response schemas.

5.12 Error handling

Domain and infrastructure methods reject invalid state with specific exceptions where the caller cannot continue, such as attempting to install a second loader or execute without a Java runtime. HTTP responses are checked before files are treated as downloaded. Deserialization verifies that required response objects exist. Installer processes check their exit codes.

Expected start failures use `ServerStartResult` rather than exceptions alone. Already running, port conflict, port-mapping failure and successful start are outcomes the interface can explain. Mod installation similarly returns a structured migration result because partial compatibility is expected when providers do not publish a replacement.

The current implementation still has inconsistent edges. Some adapters write diagnostic messages to standard output or error, and several low-level failures reach the UI as general exceptions. A unified application error model and structured logging would improve diagnosis. The report does not claim that every external failure has a specialized message, only that the principal workflows distinguish the expected cases.

CHAPTER 6

VALIDATION

Validation was performed against the complete application rather than against isolated adapters. The objective was to reproduce the sequence a user follows from an empty installation: obtain Java runtimes, create vanilla servers, add mod loaders and mods, migrate their configurations, execute them and finally remove all managed content. Running the same sequence on three computers also checked that the persisted paths and Windows-specific integrations did not depend on a single development machine.

The tests described in this chapter are scenario-based functional tests. They establish that the observed workflows completed in the stated environments, but they are not a performance benchmark and do not constitute proof that every provider release or network configuration is supported. Unless explicitly stated, a requirement is not considered independently validated merely because a related workflow succeeded.

6.1 Environment and compatibility matrix

The complete procedure was repeated on three computers. Two ran Windows 11, one with the Home edition and one with the Pro edition. The third ran Windows 10 Home. Hardware models and specifications were not recorded, so the environment comparison is limited to the operating-system information available.

Table 6.1: Operating systems used for functional validation.

Device	Operating system	Executed scenario
1	Windows 11 Home	Complete end-to-end flow
2	Windows 11 Pro	Complete end-to-end flow
3	Windows 10 Home	Complete end-to-end flow

The same component combinations were used on every device. Java 21 was obtained from Eclipse Adoptium as a compressed package, while Java 25 was obtained from

Mojang through its manifest distribution. Two server lines were exercised: Minecraft 1.21 with Forge and Java 21, and Minecraft 26.1 with Fabric and Java 25. The latter was subsequently migrated to Minecraft 26. Mods were obtained from both CurseForge and Modrinth.

Table 6.2: Software combinations exercised during validation.

Minecraft	Selected Java	Initial loader	Additional operation
1.21	Adoptium 21	Forge	Loader switch and mod migration
26.1	Mojang 25	Fabric	Loader switch, then downgrade to 26

6.2 Functional validation

6.2.1 Testing method

During development, each new or modified feature was tested by running the application and exercising the corresponding workflow. The first input was a representative value, such as Minecraft 1.20 when working on version handling. Boundary or unusual values were then selected to reveal assumptions hidden by the common case. For Minecraft versions, examples included the recent 26.1 release and the old 1.0 release. This policy was exploratory rather than a fixed automated test suite, but it was applied throughout implementation and defect correction.

The final validation used a fixed twelve-step flow on each computer. It combined operations so that the output of one step became the precondition of the next. This was important for migration testing because a successful catalogue query alone does not show that installed loader and mod state can be carried into a different configuration.

6.2.2 End-to-end procedure and results

1. **Java installation.** Java 21 was downloaded from Eclipse Adoptium and Java 25 from Mojang. Both installations completed successfully and appeared in the Java management interface. This exercised the compressed-package and manifest installation paths.

2. **Automatic Java selection.** The server Java setting was changed to automatic. This allowed each server to select the installed major version required by its Minecraft release rather than assigning one runtime manually to every instance.

3. **Server creation.** Two servers were created through the UI. One used Minecraft 26.1, which selected Java 25, and the other used Minecraft 1.21, which selected Java 21. Both server definitions and their directories were created successfully.

4. **Mod-loader installation.** Fabric was added to the Minecraft 26.1 server and Forge to the Minecraft 1.21 server. Both loaders were installed successfully through the server editor.

5. **Mod acquisition and dependency resolution.** Mods were downloaded from CurseForge and Modrinth. The selected set included projects with known required dependencies. In particular, Sodium Extra was installed and its required base mod, Sodium, was resolved and added correctly. This verified that a direct user selection can produce additional compatible installations through `ModManagerService`.

6. **Mod-loader switching.** The mod loaders were switched. The application removed loader-specific mods for which the new environment was not valid, including Fabric API, while migrating mods that publish releases for both loader types, including Sodium. The resulting changes were reported to the user.

7. **Minecraft version migration.** The Minecraft 26.1 server was downgraded to Minecraft 26. The loader and installed mods were updated to releases compatible with the target version. The server was executed successfully before and after the migration, confirming that the resulting installation remained runnable.

8. **Property editing.** Server properties were changed through the graphical editor. The modified values were present when the servers were subsequently executed, showing that the submitted model had been written to `server.properties`.

9. **Independent execution and commands.** Each server was started independently. Both reached successful execution with the Java runtime selected by the automatic policy. Commands were then submitted through the execution interface. The `say hello` command displayed the expected message in the in-game chat, and `gamerule keepInventory true` changed whether players retained their inventory after death. Finally, the `stop` command was submitted through the console instead of using the stop action in the UI. The Minecraft process ended normally, and Wave Launcher detected the external process termination and updated the session state accordingly.

10. **Port-conflict handling and external access.** Start was requested while another server was already using the configured port. In both server configurations, Wave Launcher refused the second start and informed the user that the port was occupied. This confirmed the conflict check required by FR-14. During normal execution, the router also showed the server port as forwarded, and a player successfully joined from an external network. Together, these observations verified automatic port mapping and external reachability in the tested network environment.

11. **Server deletion.** Both servers were removed through the application. The Wave Launcher servers directory was inspected afterwards and found to be empty.

12. **Java removal.** Java 21 and Java 25 were removed after their dependent servers had been deleted. The managed Java directory was inspected afterwards and found to be empty.

6.2.3 Requirement coverage

The scenario directly exercised viewing, creating, deleting and executing server instances; submitting commands; stopping a server from its console; adding and changing

loaders; acquiring mods from both supported providers; resolving dependencies; changing Minecraft versions; managing Java installations; editing properties; detecting port conflicts; and connecting from an external network through an automatically forwarded port. The `say hello` command confirmed that commands entered in Wave Launcher reached the Minecraft server, while `gamerule keepInventory true` confirmed that a command could change the running world's rules. Submitting `stop` also showed that the launcher detected a server ending through its own console rather than through the UI. The router displayed the port mapping while the server was running, and an external player joined successfully. These observations provide direct evidence for FR-01 to FR-08, FR-10, FR-12, FR-13 and FR-14.

The flow caused loader-specific mods to be removed during migration, but it did not record a separate user-initiated removal test for FR-09 or FR-11. No pair of mods with a declared incompatibility could be found during validation, so the edge case in which the application rejects two mutually incompatible mods was not exercised. Required dependency resolution was tested successfully, but this result does not provide evidence for the incompatible-mod path.

All twelve steps completed on all three computers. In addition, no application errors were encountered during the user-evaluation sessions conducted later. The results of those sessions, including the comparison with manual server setup, are intentionally reserved for Section 6.3.

6.3 Comparison with manual server setup

Three participants with different technical backgrounds were asked to create a modded Minecraft server manually and then repeat the task with Wave Launcher. Their names are omitted, and they are identified as Subjects A, B and C. Subject A was not familiar with computers or server creation. Subject B worked in IT, understood the general purpose of a Minecraft server and reported having created servers as a child, although they no longer played the game. Subject C did not work in IT but played Minecraft regularly through the official launcher. This made Subject C familiar with the game while still fitting the project's casual-player profile, since the official launcher hides most runtime details and the participant had never created a server.

This was a small observational evaluation intended to reveal differences in the process and the obstacles encountered by each participant. The manual task was always performed first, so the Wave Launcher results may include some learning carried over from the first attempt. However, the second task still required the participants to understand a different interface and workflow.

6.3.1 Test procedure

For the manual task, participants received a short list of goals rather than detailed instructions:

1. Create a server folder.
2. Download the Minecraft server file and place it in that folder.
3. Download Forge and install it in the server.

4. Download mods from Modrinth.
5. Place the downloaded mods in the server files.
6. Configure the difficulty as `hard` and the game mode as `spectator`.
7. Open the required router ports.
8. Start the server and resolve any dependencies.

No further explanation was given at the beginning. Participants could search for written or video tutorials and were also allowed to use an LLM. Assistance was provided only when a participant was severely stuck, or when an incorrect step would have prevented the remaining tasks from being completed while the participant believed it was correct. The intervention was recorded as part of the observation rather than treated as an independent completion.

Forge and Modrinth were selected to keep every participant's goal comparable. The choice was not based on an expected usability advantage, and the same broad process would have applied to Fabric and CurseForge. Specifying the provider and loader also avoided spending test time comparing ecosystems. The difficulty and game-mode values were chosen so that each participant had to locate and edit values inside the long `server.properties` file rather than leaving the defaults unchanged.

The Wave Launcher task expressed the same intended result in four user-level steps:

1. Create a server with difficulty set to `hard` and game mode set to `spectator`.
2. Add Forge.
3. Add mods.
4. Start the server.

The difference in the number and type of steps already illustrates part of the abstraction provided by the application. The manual list exposes file placement, loader installation, configuration-file editing, router setup and dependency resolution as separate responsibilities. The Wave Launcher list describes the result the player wants, while the application carries out most of those technical operations internally.

All timestamps were measured from the beginning of the corresponding task and have been standardized as minutes and seconds in Table 6.3. The reduction column compares each participant's Wave Launcher time with their manual time.

6.3.2 Manual creation across participant backgrounds

Subject A completed the folder and vanilla-server download but initially executed the JAR from the Downloads directory. Minecraft generated its files there, after which the participant moved some files into the intended server folder without moving the executable itself. Command-line directory navigation caused further difficulty, although the participant found and edited `eula.txt` without assistance. By 05:00, the vanilla files had been moved into the server folder.

Table 6.3: Completion times for manual setup and Wave Launcher.

Subject	Background	Manual	Wave Launcher	Time reduction
A	Limited computer experience; no server experience	18:39	05:00	73.2%
B	IT background; previous server experience	22:14	02:08	90.4%
C	Regular Minecraft player; no server experience	18:23	04:05	77.8%
Mean		19:45	03:44	81.1%

The Forge stage exposed version and directory problems. Subject A downloaded the latest Forge installer without first comparing it with the selected Minecraft version, consulted two tutorials and created an unnecessary nested Forge directory. The participant then attempted to run the server from that directory. While browsing Modrinth, they first selected ETF for Fabric, noticed the loader mismatch, and replaced it with a compatible TerraBlender release. Assistance was needed to explain that the mod belonged in a `mods` directory. The participant later found and correctly changed the difficulty and game-mode entries, but needed to be told to enter `stop` before restarting the server. Port forwarding was unfamiliar and required direct explanation. The final Forge server was running at 18:39.

Subject B approached downloads more cautiously because of their IT background. They checked whether `minecraft.net` was a legitimate source and scanned the Forge JAR with VirusTotal. They also compared the Forge and Minecraft versions before installation. Even with this background, uncertainty remained about the required order: the participant questioned whether the vanilla server should be run before Forge and initially started searching for mods before installing the loader.

Compatibility selection took a noticeable part of Subject B's test. Create (the first selected mod) was unavailable for Minecraft 26.2, Sodium did not provide a Forge release for the chosen combination, and the participant eventually selected Xaero's Minimap. They searched for instructions, installed Forge at 12:33 and, like Subject A, created an unnecessary nested folder. The mod was placed in that nested installation. The configuration properties were changed correctly after an online search. Router configuration then required searches for the gateway, local IP address, protocol, Minecraft port and forwarding settings. Assistance was needed to identify the port-forwarding page and to clarify that the internal and external ports should match. At the end, the participant read the process output but needed to be told that Minecraft's `Done` line indicates successful startup. Completion was recorded at 22:14.

Subject C moved through the early download steps quickly. They created the folder, downloaded the vanilla server by 00:40, compared Forge versions and obtained the recommended installer by 01:11. They then searched for instructions on installing a loader and used the batch command provided on the Minecraft download page to start the vanilla server. EULA acceptance was completed at 04:14. Assistance was needed at 06:30 to explain that the Forge installer itself had to be executed. After that clarification, the participant selected the server-install option and chose the correct directory.

Subject C filtered Modrinth by Minecraft version, downloaded Xaero's Minimap and

needed clarification that it belonged in a `mods` subdirectory. They correctly changed both requested properties by 11:00. Port forwarding was again the largest unfamiliar area. The participant found the default gateway with `ipconfig`, but required an explanation of the router interface and the port to open. When starting the completed server, they first launched the vanilla JAR, which excluded Forge and its mods. They were then directed to the Forge-generated batch file. At 18:23, they still could not determine from the log whether startup had succeeded and needed to be shown the `Done` line.

The three manual attempts show both differences and similarities between skill levels. Subject A had the most difficulty with directory organization and command-line navigation. Subject B performed more security and compatibility checks and understood more of the terminology, but this did not produce the shortest completion time. In fact, the additional verification and online research made Subject B's manual attempt the longest. Subject C was faster in the first download stages because of familiarity with Minecraft, but that familiarity did not extend to loader execution, mod directories, networking or server logs.

The manual completion times require an important qualification. Subject B's basic understanding of servers allowed them to attempt the port-forwarding process rather than stopping immediately. They independently searched for the router gateway, local IP address, required protocol and Minecraft port before receiving limited clarification about the router settings. Subjects A and C reached an impasse at this stage and did not know how to continue, so they were directly told how to configure the forwarding rule. This assistance allowed their tests to proceed sooner. Subject B's longer manual time therefore does not indicate poorer performance than Subjects A and C. Part of the difference comes from spending more time attempting the most technical step independently, while the other two participants received the procedure after becoming stuck. For this reason, the recorded times describe the observed sessions but should not be used alone to rank the participants' manual ability.

All three participants eventually encountered uncertainty about the relationship between the vanilla server, Forge and the selected mods. Subjects A and C needed help identifying the mod directory, while Subject B searched for instructions and initially proceeded before the loader was installed. Subjects A and B created unnecessary nested Forge directories, and Subject C first executed the vanilla server instead of the Forge launch script. One possible reason for the nested directories is the Forge installer's warning when the selected destination already contains files. The warning is displayed in red text, which can reasonably be interpreted as an error even though selecting the existing server directory is normal and required. Creating a new empty subdirectory may therefore appear to the user as the safer option. The observations do not establish that this warning was the sole cause, but the same behavior in two participants suggests that the installer feedback may contribute to the confusion. Overall, these results indicate that general computer knowledge and familiarity with Minecraft help with individual actions, but neither one alone gives a clear mental model of the complete modded-server installation.

6.3.3 Wave Launcher use across participant backgrounds

With Wave Launcher, Subject A created the server, selected a version, accepted the EULA and configured the requested difficulty and game mode. They asked about the

message of the day, execution flags and Java selection, which indicates that some labels still assume technical knowledge. After creation, the participant correctly returned to the editor and selected Forge. They first used the installed-mod filter instead of the add-mod button, but after being shown the correct button they added TerraBlender through CurseForge and started the server. The complete task took 05:00.

Subject B selected Minecraft 1.21.1, the requested game mode and difficulty, accepted the EULA and manually chose an existing Java installation, although automatic selection was sufficient. The participant added Forge by 01:00. They were unsure which Forge release to select and were advised to choose the latest compatible option. In the mod popup, they initially searched without selecting a supplier. Once this requirement was explained, they found and added Hydraulic Machines, saved the server and started it at 02:08.

Subject C created and configured the server, then paused at 00:54 to search online for the meaning of execution flags. They opened the existing server at 01:30, selected Forge 65.1.2 and, like Subject A, first tried the installed-mod search field rather than the add-mod button. After being redirected to the popup, they selected Modrinth, found a Xaero's Minimap, selected a version and added it by 03:43. The changes were saved at 04:02 and the server was running at 04:05.

Background had less effect on completion with Wave Launcher than it did on the path taken during manual creation. Subject B was the fastest, which is consistent with greater familiarity with software configuration, but Subjects A and C also completed the workflow in five minutes or less. The difficulties were mostly local interface-discovery issues rather than server-administration problems. Subjects A and C confused the installed-mod filter with the add-mod control. Subject B attempted a search before choosing a supplier. Subjects A and C questioned execution flags, and Subject A also needed clarification about Java selection and the message of the day.

These observations reveal useful areas for UI improvement. The add-mod action could be made more prominent, and the supplier requirement could be explained before a search is attempted. Optional technical fields such as execution flags could include short descriptions or remain collapsed for casual users. Java's automatic-selection behavior should also be clearer so that users do not assume they must understand or select a runtime manually. None of these issues prevented completion once the relevant control was identified.

6.3.4 Manual setup compared with Wave Launcher

Wave Launcher reduced the mean completion time from 19:45 to 03:44, an observed reduction of 81.1%. The individual reductions ranged from 73.2% to 90.4%. With only three participants and a fixed task order, these values should be read as results of this evaluation rather than as estimates for the wider population. Even so, the time difference is supported by a visible reduction in both the number of operations and the amount of external research.

Some activities were common to both methods. Participants still had to choose a Minecraft version, a loader and a mod, approve the EULA, configure the requested gameplay properties and start the server. Compatibility concepts also remained visible to some extent, since users selected Forge and a provider. The difference was where responsibility for the consequences of those choices was placed. During manual creation,

participants had to obtain the correct artifacts, decide where they belonged, use the correct launch file and interpret provider and server output. In Wave Launcher, these actions were coordinated behind the selected options.

The largest manual difficulties were network configuration and determining whether the server had started correctly. Every participant needed help with port forwarding or with specific router values. Two participants could not independently recognize successful startup in the logs, and the remaining participant also needed guidance to search for the `Done` message. Mod and loader compatibility created a related problem: participants selected unavailable or incompatible combinations, installed files in the wrong order, or launched the vanilla server without the loader. A manual failure at any of these points left the participant to determine whether the problem came from Minecraft, Java, Forge, a mod, a missing dependency, the directory layout or the network.

Wave Launcher directly addresses these two main obstacles. It requests and releases the router mapping when the server session starts and stops, checks for a local port conflict, and reports mapping failure as a startup result. The validation described earlier also confirmed a visible router mapping and a successful connection from an external network. For execution, the application starts the correct vanilla or loader JAR with the selected Java runtime and tracks the resulting process, so the UI can represent whether a session is running. Loader and mod searches are filtered by the selected Minecraft environment, required dependencies are resolved automatically, and migration results report components that cannot be retained. The application therefore does more than shorten the manual instructions. It takes responsibility for the two areas that caused the most uncertainty in all three observations: network exposure and the combined interpretation of startup, loader, mod and dependency state.

6.4 Known defects and limitations observed during testing

No known reproducible defects remain at the time of writing. Errors found during development and exploratory testing were corrected before the final three-device procedure. This statement describes the current observed state rather than claiming that the program is defect-free. The test matrix is finite, provider data can change, and hardware or network configurations outside the three computers may reveal additional failures.

Java installation is limited to compressed packages, such as ZIP and TAR.GZ archives, and manifest-based distributions. Eclipse Adoptium also publishes MSI installers, but that installation path was excluded after it introduced disproportionate complexity during development. Wave Launcher therefore obtains Adoptium runtimes through portable compressed packages and Mojang runtimes through manifests. This avoids system-wide installer behavior but means that a provider release available only as an MSI cannot be installed by the current adapters.

Loader support is limited to Forge and Fabric. Other active ecosystems, including NeoForge and Quilt, were not implemented. They are more commonly selected by experienced or power users, whereas Wave Launcher is scoped around the needs of casual players. The port and adapter design allows another loader catalogue and installer to be added later.

Finally, successful execution on Windows 10 Home, Windows 11 Home and Windows 11 Pro does not establish support for other operating systems. The interface uses .NET MAUI, but process execution, device information and the validated network behavior remain Windows-specific in the current release.

CHAPTER 7

CONCLUSIONS AND FUTURE WORK

7.1 Conclusions

Wave Launcher grew out of a simple observation: setting up a private Minecraft server still requires more technical knowledge than many players have or want to acquire. The project responds to that problem with a local Windows application that coordinates the server software, Java runtimes, Forge or Fabric, mods from Modrinth or CurseForge, configuration files, process execution and router port mappings. The result is not intended to be a general hosting panel. It is a focused desktop workflow for players who want to self-host without learning a separate administration procedure for every component.

The implementation demonstrates that the selected components can be represented behind a common application model despite substantial differences between their providers. Mojang and Adoptium distribute Java in different forms. Forge exposes Maven metadata while Fabric provides its own catalogue endpoints. Modrinth and CurseForge use different identifiers, authentication rules and response schemas. Mapping them into domain types makes server creation and migration an application operation rather than a sequence of unrelated downloads.

Hexagonal architecture was valuable during this work because external integrations could be implemented and revised independently. The later development of the interface did require service changes, but those changes occurred through explicit contracts rather than direct dependencies from pages to files or HTTP endpoints. JSON persistence also proved proportionate to a single-user desktop application, while repository interfaces retain a path toward stronger storage if future scope demands it.

The iterative weekly process suited a project whose uncertainty lay mainly in external interfaces and a new UI framework. The high-level schedule remained stable through functional completion. The final correction phase grew by one week due to MAUI form behavior, illustrating why short milestones and time reserved for integration were necessary.

The functional validation strengthens this conclusion beyond successful imple-

mentation alone. The same end-to-end scenario completed on Windows 10 Home, Windows 11 Home and Windows 11 Pro. It covered managed Java 21 and 25 installations, automatic runtime selection, Minecraft 1.21 and 26.1 servers, Forge and Fabric, CurseForge and Modrinth, dependency resolution, loader switching, a downgrade to Minecraft 26, property persistence, execution, port-conflict detection and complete cleanup. No reproducible error was observed during that procedure.

The comparison with three users also provided initial evidence for the usability objective. Manual creation took between 18:23 and 22:14, whereas the Wave Launcher workflow took between 02:08 and 05:00. Mean completion time fell from 19:45 to 03:44. During manual setup, all three users struggled with port forwarding or needed specific guidance, and interpreting successful startup and loader or mod compatibility also caused repeated uncertainty. Wave Launcher automated these responsibilities, leaving mainly local interface questions such as where to add a mod or what execution flags mean. The sample is too small to support broad statistical conclusions, but the consistent result across three different backgrounds indicates that the application reduced both completion time and the technical knowledge required in the observed task.

7.2 Future work

7.2.1 Extend the empirical evaluation

The initial comparison involved only three participants and always presented the manual task first. A larger evaluation should balance task order, use a more detailed intervention protocol and include participants from a wider range of ages and technical backgrounds. It should record completion, time, interventions, errors and user confidence consistently. This would show whether the observed reduction is maintained beyond the small exploratory sample and would separate the effect of Wave Launcher from learning during the manual task. The functional matrix should also be extended as new Minecraft, Java, Forge and Fabric versions are published, with failures classified between application defects, provider outages and unsupported router environments.

7.2.2 Additional platforms

.NET MAUI and the internal boundaries leave room for macOS or Linux desktop support, but the present product should not be described as cross-platform until adapters and complete tests exist. New process-executor and device-information implementations are required. File permissions, executable discovery, archive formats and platform packaging must also be addressed. Android and iOS are not natural server-hosting targets even though MAUI can compile interface projects for them, so desktop platforms have higher priority.

7.2.3 Modpack support

The current mod integrations manage individual projects obtained from Modrinth and CurseForge. A future version could also browse and install modpacks published by these already supported suppliers. Installing a modpack would require the application

to interpret its manifest, obtain the specified loader and Minecraft version, download its mods and configuration files, and present any optional components to the user. Reusing the existing supplier adapters and compatibility model would keep modpack installation within the same server-management workflow instead of requiring users to assemble a published pack manually.

7.2.4 World migration with MCA Selector

Updating the server software does not update terrain that has already been generated in its world. Integrating MCA Selector could make world migration more useful by identifying and deleting generated chunks that have not been modified. When those areas are explored again, Minecraft would regenerate them using the terrain and features of the newer version. The launcher should present the affected area clearly and require explicit confirmation before changing the world files, since deleting the wrong chunks could remove player changes.

7.2.5 Whitelist management

Wave Launcher could provide an interface for managing the server whitelist without requiring the user to edit files or enter commands in the server console. The interface should list the currently allowed players and support adding or removing entries from the launcher. This would make access control consistent with the existing graphical management of properties, mods and execution while keeping the underlying Minecraft whitelist synchronized.

7.2.6 Network diagnostics

UPnP and NAT-PMP cover many home routers but cannot solve carrier-grade NAT, disabled discovery or firewall rules. A diagnostic screen could explain the local address, mapping protocol, public port and failure stage. Optional firewall-rule assistance and a reachability check from outside the local network would make the result easier to verify. These features must be designed cautiously because network configuration changes have security implications.

7.2.7 Distribution and community development

Wave Launcher's source code is publicly available at `https://github.com/nvl-ez/Wave`. The project is intended to be distributed as free software under a GNU copyleft licence.

APPENDIX A

USER GUIDE

This annex gives a practical overview of how Wave Launcher is used. Chapter 6 describes the environments, procedure and results used to validate the same operations.

A.1 Initial configuration

On the first launch, the user opens Settings and goes to the Java area. Mojang and Eclipse Adoptium are available as suppliers. The selected release is installed in Wave Launcher's application-data directory, which keeps it separate from system-wide Java installations.

To use CurseForge, the user must also enter an API token in the general application settings. Modrinth does not require this token.

A.2 Creating a server

From the Servers page, the user creates a new instance and supplies a name. An icon can be selected, after which the application stores a normalized copy. The Minecraft release and Java runtime are chosen from the presented lists. Snapshot visibility can be enabled when an unstable Minecraft release is required.

The General area contains launch options and EULA acceptance. The Properties area exposes the Minecraft configuration definitions through typed controls. If a modded server is desired, Forge or Fabric and one of its compatible releases is selected. Mods can then be searched from the Mods area. Submitting the form downloads and constructs the complete server directory.

A.3 Editing and migrating

Selecting an existing card opens the same editor with stored values. Ordinary settings can be changed directly. A Minecraft or loader change may require replacement or

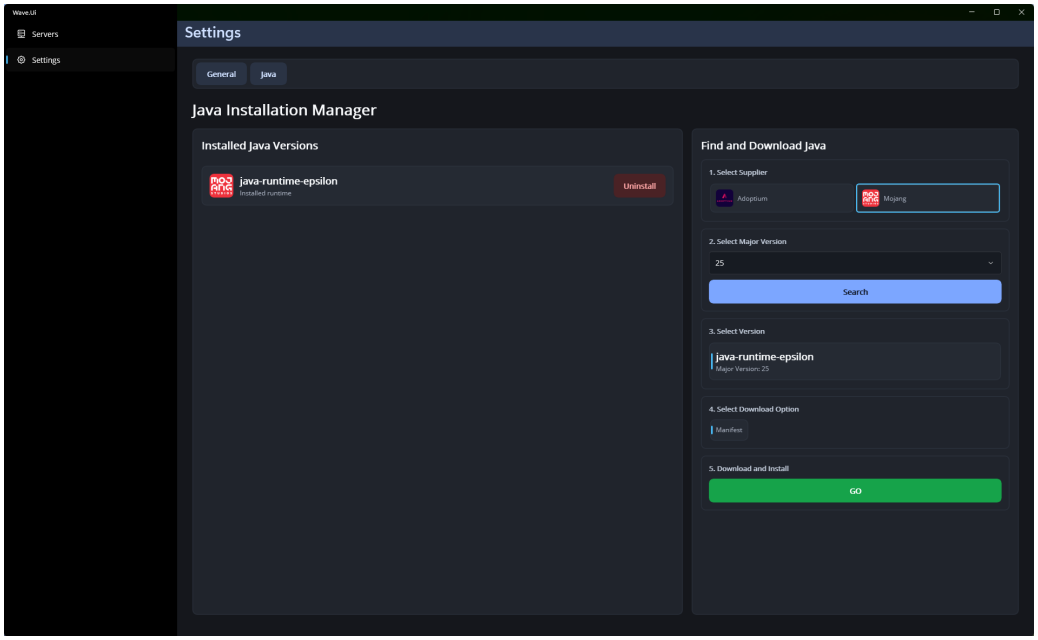

Figure A.1: Java runtime configuration.

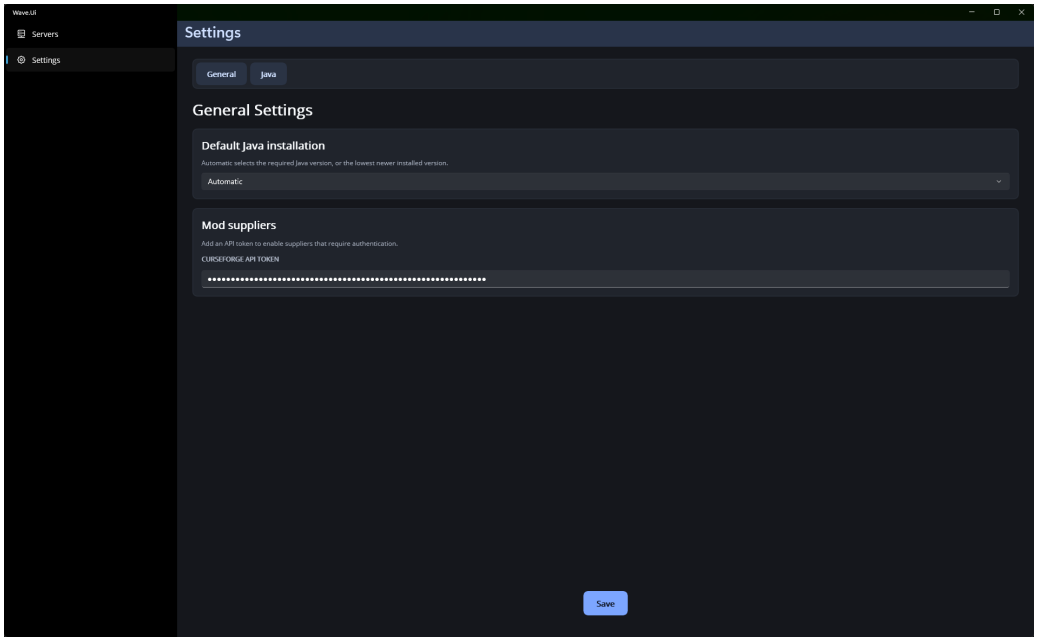

Figure A.2: Application and provider settings.

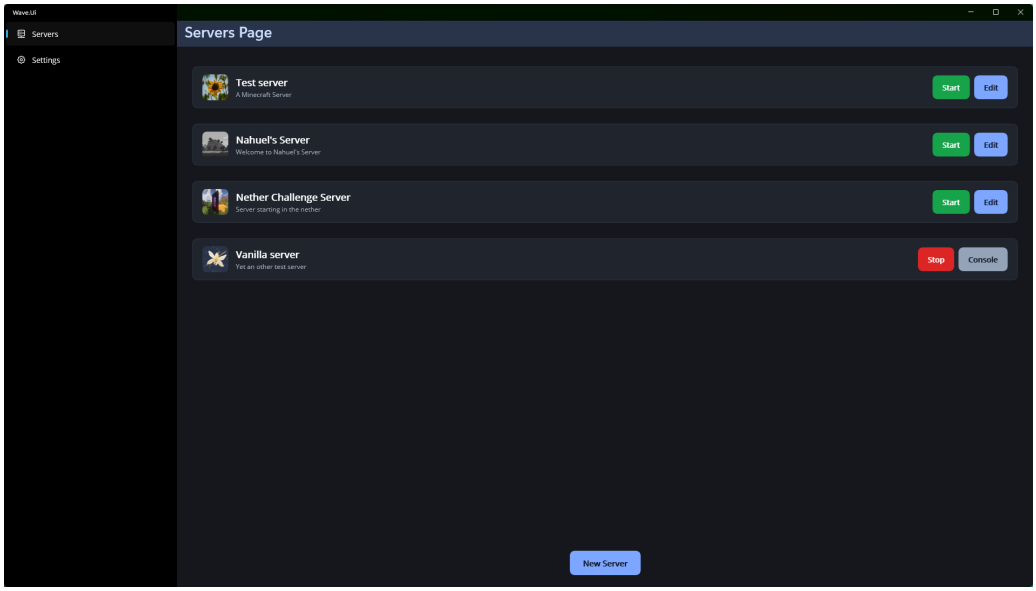

Figure A.3: Server list and new-server action.

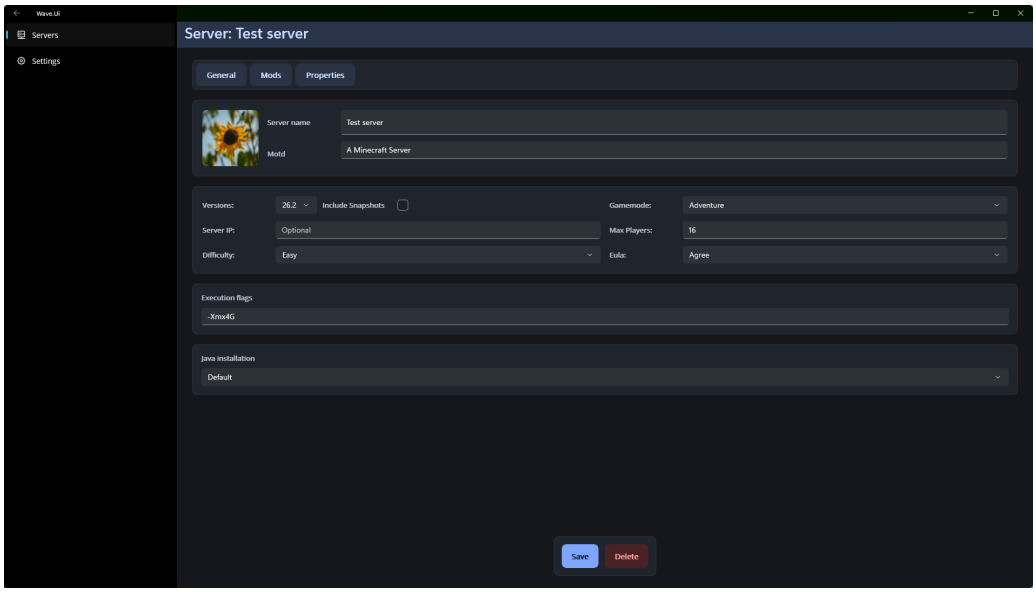

Figure A.4: General configuration for a new server.

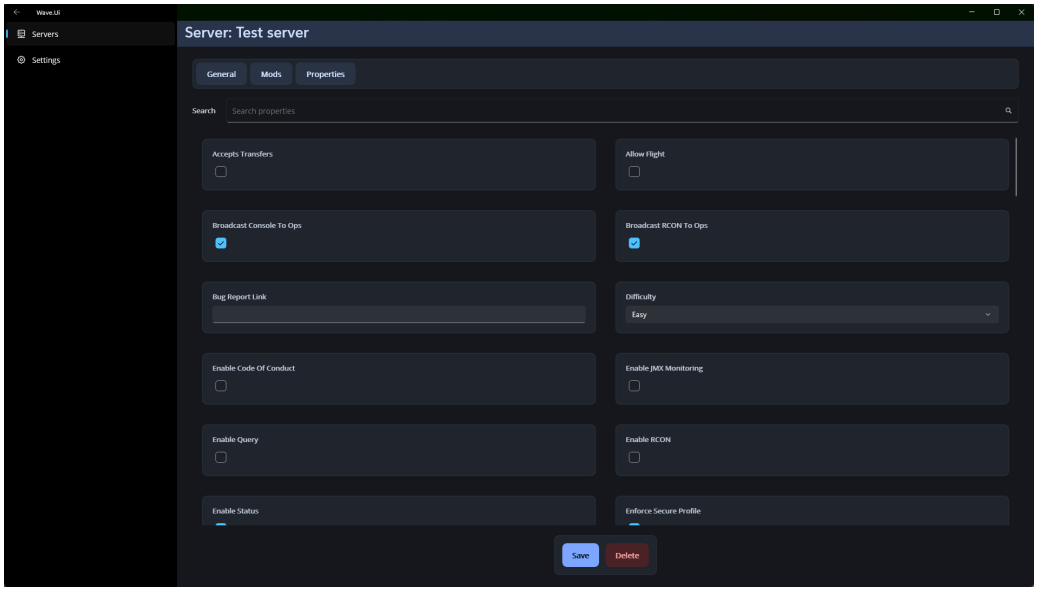

Figure A.5: Minecraft server-properties configuration.

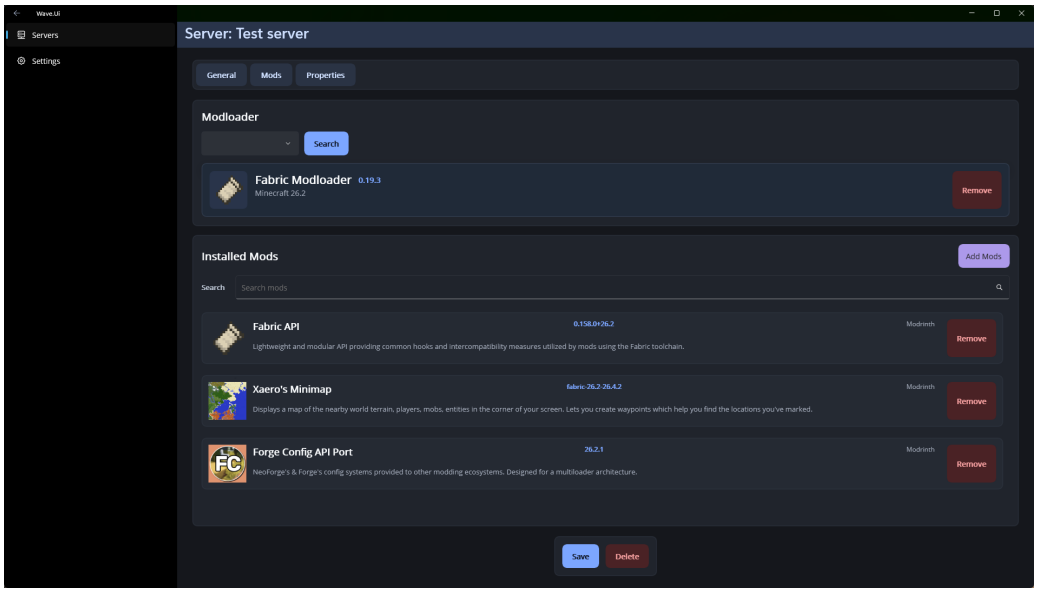

Figure A.6: Loader and installed-mod configuration.

removal of related components. Before applying the edit, the changes popup summarizes these consequences. After confirmation, Wave Launcher downloads compatible artifacts and reports mods that were required, incompatible, unavailable or failed.

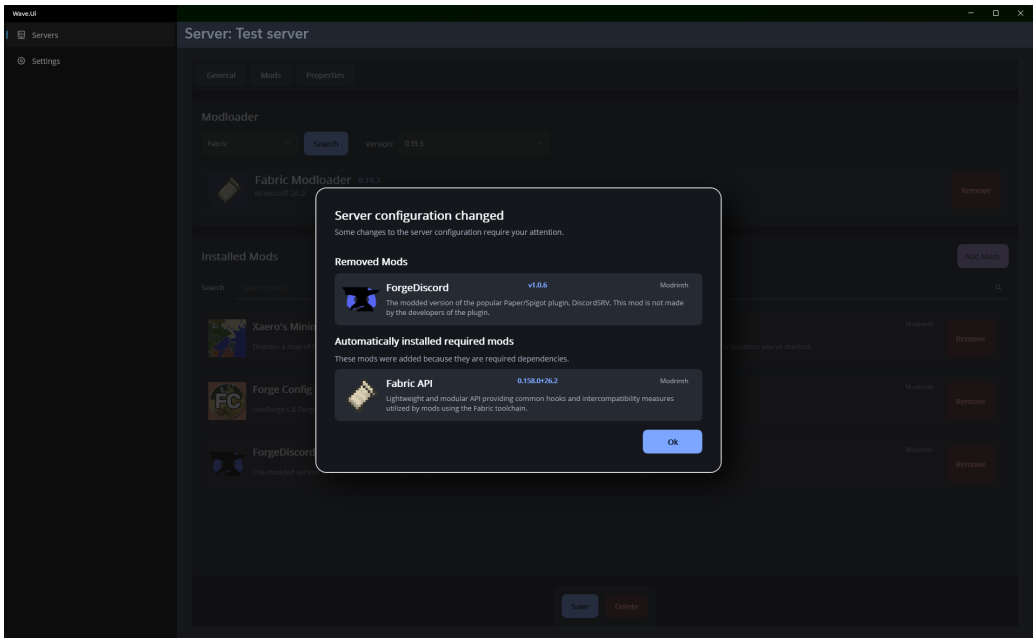

Figure A.7: Summary of changes before updating a server.

A.4 Starting and stopping

The start action checks that the selected Java runtime exists and that another Wave server is not using the configured port. The application requests an automatic router mapping and starts the Java process only if this step succeeds. The Execution page shows the redirected output. Server commands can be entered while the session is active.

The normal stop action sends the server command that allows Minecraft to save its world, closes the process session and releases the temporary port mapping. Closing Wave Launcher also ends managed running sessions.

A.5 Removing content

Mods can be deselected through the server editor. Removing a loader also removes mods that depend on it. A Java installation must first be detached from every saved server before it can be uninstalled. Deleting a server removes its managed directory, including its world and configuration, so any world that must be retained should be copied before deletion.

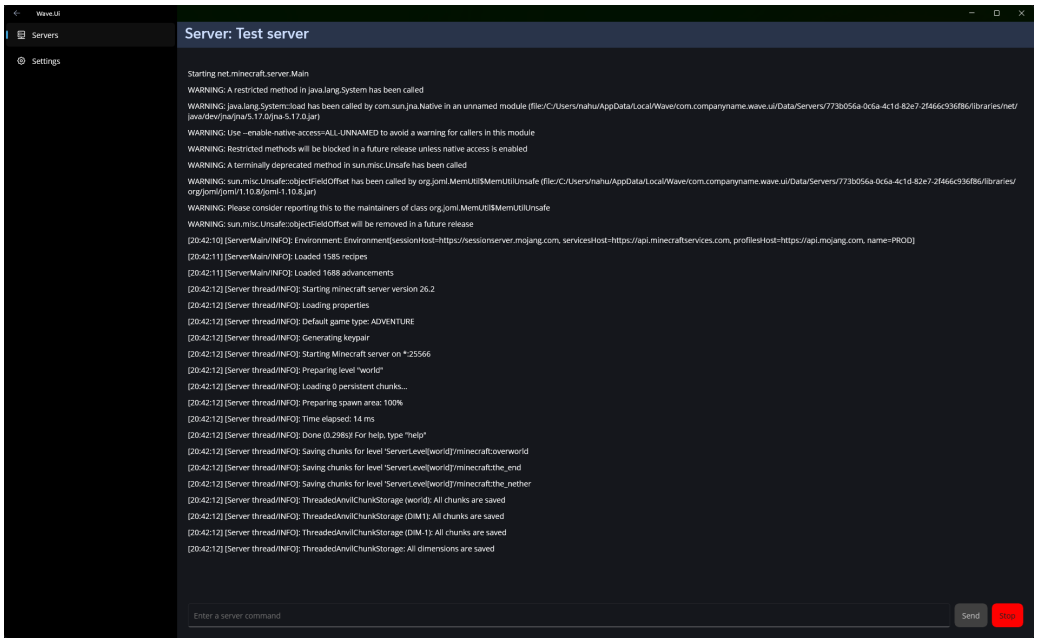

Figure A.8: Execution console for a running server.

APPENDIX B

REPOSITORY STRUCTURE

`Wave.Domain` Server, Minecraft, Java, mod, loader, device and pagination models.

`Wave.Application/In` Operations consumed by view models.

`Wave.Application/Middle` Contracts for managers used inside server workflows.

`Wave.Application/Out` Repository, provider, process, platform, image and network ports.

`Wave.Infrastructure/In` Implementations of the principal application use cases.

`Wave.Infrastructure/Middle` Version, loader, mod, properties, EULA and path orchestration.

`Wave.Infrastructure/Out` JSON persistence, external HTTP adapters, Java installers, Windows process execution and SharpOpenNat port mapping.

`Wave.Ui/Pages` Pages, views and view models organized by content area.

`Wave.Ui/Views` Reusable collections, forms, feedback and navigation controls.

`Wave.Ui/Resources` Icons, images, fonts, colors and styles packaged by MAUI.

BIBLIOGRAFIA

[1] Crafty Controller, “Crafty documentation,” https://docs.craftycontrol.com/, 2026, accessed: 2026-08-09. 2.1

[2] ——, “Crafty 4 license,” https://gitlab.com/crafty-controller/crafty-4/-/blob/master/LICENSE, 2022, accessed: 2026-08-09. 2.1

[3] ——, “Linux installation guide,” https://docs.craftycontrol.com/pages/getting-started/installation/linux/, 2026, accessed: 2026-08-09. 2.1

[4] ——, “Crafty controller,” https://craftycontrol.com/#home, 2026, accessed: 2026-08-09. 2.1

[5] MC Server Soft, “Mc server soft: Free minecraft server wrapper ui for windows,” https://www.mcserversoft.com/, 2026, accessed: 2026-08-09. 2.1

[6] ——, “Getting started with mcss,” https://docs.mcserversoft.com/, 2026, accessed: 2026-08-09. 2.1

[7] ——, “Sourcing a server file,” https://docs.mcserversoft.com/basic/create-server/sourcing-server-file, 2026, accessed: 2026-08-09. 2.1

[8] Fork, “Fork: Minecraft server manager,” https://www.fork.gg/, 2026, accessed: 2026-08-09. 2.1

[9] MCSManager, “Mcsmanager project repository,” https://github.com/MCSManager/MCSManager, 2026, accessed: 2026-08-09. 2.1

[10] ——, “Dependencies: Install java environment,” https://docs.mcsmanager.com/setup_package.html, 2026, accessed: 2026-08-09. 2.1

[11] ——, “Setup java edition server,” https://docs.mcsmanager.com/setup_java_edition.html, 2026, accessed: 2026-08-09. 2.1

[12] CubeCoders Limited, “AMP deimos 2.7.0 release notes,” https://discourse.cubecoders.com/t/amp-deimos-2-7-0-release-notes/38578, 2026, accessed: 2026-08-09. 2.2

[13] ——, “AMP: Game server control panel,” https://cubecoders.com/amp/install, 2026, accessed: 2026-08-09. 2.2